

DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE
INDIAN INSTITUTE OF TECHNOLOGY MADRAS
CHENNAI – 600036

Learning to Rank with Context Effects

A Thesis

Submitted by

AJAZ HUSSAIN

For the award of the degree

Of

MASTER OF TECHNOLOGY

© Indian Institute of Technology Madras

CONTENTS

Page

CHAPTER 1 CONTEXT EFFECTS IN CHOICE AND RANKING 1

1.1 Choices Depend on What Else Is on the Table 1

1.1.1 Similarity Is Judged in Context 1

1.1.2 Context Effects in Choice 2

1.2 From Choosing to Ranking 3

1.3 Where Independent Scoring Breaks Down 4

1.4 Choice Models: MNL, the Universal Logit and IIA 4

1.4.1 The Multinomial Logit 5

1.4.2 The Universal Logit 5

1.4.3 Independence of Irrelevant Alternatives 5

1.5 The Context-Dependent Random Utility Model 6

1.5.1 The Idea: Every Item Plays Two Roles 6

1.5.2 From the Universal Logit to the CDM 6

1.5.3 Low-Rank CDM and Identifiability 7

1.5.4 What the CDM Cannot Do 7

1.6 The Linear Context Logit 7

1.6.1 The Model 8

1.6.2 Why LCL? 8

1.7 CDM and LCL Side by Side 9

1.8 Summary 9

CHAPTER 2 SELF-ATTENTION AS A MODEL OF CONTEXT 11

2.1 The Idea Behind Self-Attention 11

2.2 The Architecture 12

2.2.1 Self-Attention for a Choice Set 12

2.2.2 The Encoder Block and the Readout 12
2.2.3 Properties Relevant to Choice 13
2.3 How Self-Attention Evaluates a Choice Set: 13
2.4 Can Self-Attention Express Context Effects? 14
2.4.1 Attention Logits as CDM-Style Pairwise Interactions 14
2.4.2 Attention Can Compute the LCL Set Summary 14
2.4.3 Why a Multiplicative Interaction Is Needed 14
2.4.4 Expressivity Is Not Identification 15
2.5 Prior Work: Set-Aware Neural Models for Ranking and Choice 15
2.5.1 Context-Aware Learning to Rank with Self-Attention 15
2.5.2 FETA and FATE: Learning Context-Dependent Choice Functions 16
2.6 Motivation: Testing Recovery, Not Just Accuracy 17
2.7 Summary 19

CHAPTER 3 EXPERIMENTAL DESIGN: TESTING RECOVERY ON LCL DATA 20

3.1 Overview of the Experiment 20
3.2 Generating the Data 21
3.2.1 Ground Truth and Reference Setting 21
3.2.2 Simulated Choices and Reference Losses 21
3.2.3 Splits, Seeds and Worlds 22
3.3 Models 23
3.3.1 The Self-Attention Model 23
3.3.2 Mean-Pool and Flat Networks 24
3.3.3 Oracle and Two Small Attention Models 25
3.4 Training and Hyperparameters 26
3.5 Sweeps 27
3.6 Evaluation Metrics and the Probe 27
3.7 Limits and Reproducibility 29

CHAPTER 4 RESULTS 31

4.1 Training Behaviour 31
4.2 Reference Setting: Accuracy Versus Recovery (RQ1) 33

4.2.1 Prediction 33
4.2.2 Recovery of the context matrix 35
4.2.3 No false context effects 36
4.2.4 How LCL-shaped are the models? 36
4.2.5 Does accuracy imply recovery? 37
4.3 What Decides Recovery (RQ2) 38
4.3.1 Training-Set Size (E2) 38
4.3.2 Rank of the Context-Effect Matrix (E3) 39
4.3.3 Sparsity of the Context-Effect Matrix (E4) 40
4.3.4 Number of Features (E4) 40
4.3.5 Depth of the Network (E5) 41
4.3.6 Random Initialisation (E6) 43
4.3.7 Optimisation Settings (E7) 43
4.3.8 Recovery of the Preference Vector 44
4.4 Behaviour on Shifted Choice Sets (RQ3) 45
4.5 Attention Weights Versus Behavioural Probing 46
4.6 Summary of Findings 47

CHAPTER A Planted Truth and Hyperparameter Tuning **49**

BIBLIOGRAPHY **51**

CHAPTER 1

CONTEXT EFFECTS IN CHOICE AND RANKING

This chapter tells the first half of the story. It starts from everyday experience, where what we choose depends on what else is on offer, and follows that observation into ranking systems, which are built on the opposite assumption. It then shows why that assumption fails and builds up, step by step, the two statistical models that describe context effects explicitly: the Context-Dependent Random Utility Model (CDM) and the Linear Context Logit (LCL). Both are interpretable, and both have parameters that we can plant in synthetic data and later try to recover. That second point is what the rest of the thesis depends on.

1.1 CHOICES DEPEND ON WHAT ELSE IS ON THE TABLE

Most of us feel that we know what we like. Ask someone which of two hotels they prefer and they will give an answer, and we tend to assume that the answer would not change if a third hotel appeared on the page. Psychology has shown for decades that it often does. The same two options can be judged differently depending on what else is present. We call any such dependence a **context effect**.

1.1.1 Similarity Is Judged in Context

The idea goes back to Tversky's account of similarity (Tversky, 1977). He argued that people do not compare objects as points in a fixed space. They compare them as bundles of features, and how much a feature counts depends on how useful it is for telling apart the objects *currently under consideration*. Its weight is not a fixed property of the feature; it is set by the group it appears in.

His experiments make the point sharply. Subjects saw a neutral schematic face a, a frowning face b and a smiling face c, plus a fourth face that was either smiling (Set 1) or frowning (Set 2). They were asked which of the other faces was most similar to a. The frowning face b was chosen by 44% of subjects in Set 1 and by 80% in Set 2. The faces a, b and c were identical in the two conditions; only the fourth, seemingly irrelevant face changed, and b's similarity to a moved by 36 percentage points. Goldstone, Medin and Halberstadt later showed that the weight of a dimension follows how much it varies within the displayed set, and that a dimension can be switched off altogether when the other items do not vary on it (Goldstone et al., 1997).

1.1.2 Context Effects in Choice

The same thing happens when the task is not rating similarity but *choosing*. Three effects are classic, and we will meet all of them again.

- **The attraction (decoy) effect.** Take two laptops: A is cheap and slow, B is faster and pricier. Now add a third, B', that looks like B but is a bit slower and a bit more expensive. B' is worse than B on every count, so almost nobody picks it, yet B now looks like a bargain and its share goes up. An option nobody wants changed who wins (Huber et al., 1982).

- **The compromise effect.** An option sitting between two extremes gains appeal simply because it is the middle one. A streaming service offering Basic at \$8, Standard at \$13 and Premium at \$18 will find many people settling on Standard. Remove Premium and Standard suddenly feels like the expensive choice, and Basic gains share (Simonson, 1989). (The prices are only illustrative.)

- **The similarity effect.** A new option takes share mostly from the options most like it (Tversky, 1972). The textbook illustration is the red-bus/blue-bus problem (Debreu, 1960): a traveller is indifferent between a car and a red bus, so each gets half the choices. A blue bus, identical in everything but colour,

is added. Intuitively the buses split their half and the car keeps its half. A model with fixed utilities instead gives each of the three options a third, so the car loses a sixth of the market to a vehicle that is only a repaint.

What these have in common is that the *relative* appeal of two items depends on a third. That is the property we need to track as we move from psychology to ranking systems.

1.2 FROM CHOOSING TO RANKING

Everyday choices become an engineering problem the moment a system has to present a list. A hotel site, an online shop or a search engine holds many candidate items for a user and has to decide which to show first. The standard recipe is **learning to rank** (LTR). Each item i in the candidate set C is described by a feature vector $x_i \in \mathbb{R}^d$ (price, star rating, review score and so on). A model is trained on past behaviour, such as which hotel was clicked or booked, to predict how likely each item is to be chosen. At serving time it assigns a score $s_i = f(x_i)$ to each candidate and sorts the list so that the most likely choice is on top.

Classical LTR methods differ in how they train f. RankNet learns from pairs of items, LambdaRank scales its gradients by the change in NDCG that a swap would cause, and LambdaMART combines those gradients with boosted trees (Burges, 2010). LambdaMART remains a strong classical baseline. They all share one property, though: the score of item i is a function of x_i alone. What else is on the page has no influence on it.

This is a very natural design, and it amounts to a model of the user. Every item has a fixed appeal, and the user picks whichever has the highest. Section 1.1 suggests that this is not how people behave.

1.3 WHERE INDEPENDENT SCORING BREAKS DOWN

Go back to the streaming plans. A ranker that scores each plan on its own would give Standard the same score whether or not Premium is listed. People do not behave that way: with Premium present Standard looks like the sensible middle, and without it Standard looks dear. The best ordering for one candidate set is not necessarily the best for another, and a model that scores items one at a time cannot see the difference.

This is not only a laboratory curiosity. Tomlinson and Benson studied 276,593 hotel bookings from Expedia and found statistically significant context effects in how people weigh features (Tomlinson and Benson, 2021). When the hotels shown had a high mean star rating, users became *less* price-sensitive, and when they had a high mean review score, users became *more* price-sensitive. The same user, facing a different set of options, behaves as if they had different preferences.

So if we want to predict what someone will choose from a set, and rank accordingly, we have to let an item's appeal depend on the other items in the set. The next question is how to describe that dependence precisely. For this we turn from ranking to the older language of discrete choice.

1.4 CHOICE MODELS: MNL, THE UNIVERSAL LOGIT AND IIA

In a discrete choice setting a decision maker is shown a **choice set** (or **slate**) $C \subseteq \mathcal{U}$ drawn from a universe of items $\mathcal{U}$ and picks one item. A **choice model** gives the probability $\Pr(i, C)$ that item $i \in C$ is picked, with $\sum_{i \in C} \Pr(i, C) = 1$. This is the same prediction problem as in LTR; ranking by $\Pr(i, C)$ puts the most likely choice on top.

1.4.1 The Multinomial Logit

The workhorse is the multinomial logit (MNL) (Luce, 1959; McFadden, 1974). Each item has a fixed utility $v(i)$ and

$$\Pr(i, C) = \frac{\exp(v(i))}{\sum_{j \in C} \exp(v(j))}. \tag{1.1}$$

When items have features, the utility is typically linear, $v(i) = \theta^\top x_i$ (the *conditional logit*), with a single preference vector $\theta \in \mathbb{R}^d$ shared by all choice sets. This is exactly the "fixed appeal" model behind independent scoring: $s_i = \theta^\top x_i$ and the chance of choosing i rises with it.

1.4.2 The Universal Logit

The MNL is one point in a much larger family. Any choice model that gives every item a positive probability can be written in the same softmax form if we let the utility depend on the whole set,

$$\Pr(x, C) = \frac{\exp\big(v(x \mid C)\big)}{\sum_{y \in C} \exp\big(v(y \mid C)\big)}, \tag{1.2}$$

because we can always take $v(x \mid C) = \log \Pr(x, C)$. This is called the **universal logit**. It is completely general and therefore useless to estimate directly, since it needs a separate utility for every item in every possible set. The MNL is the special case $v(x \mid C) = v(x)$, where the set plays no role. Every model of context effects sits between these two extremes, and the question each one answers is how much structure to put on $v(x \mid C)$.

1.4.3 Independence of Irrelevant Alternatives

In the MNL, taking the ratio of two probabilities makes the denominator cancel:

$$\frac{\Pr(i, C)}{\Pr(j, C)} = \exp\big(v(i) - v(j)\big) \qquad \text{for every } C \ni i, j. \tag{1.3}$$

The odds of choosing i over j do not depend on what else is in the set. This is the **independence of irrelevant alternatives** (IIA). It is what makes the MNL easy to estimate, and it is also what makes it blind to the red bus and the blue bus, the decoy laptop and the missing Premium plan. Each of those examples is a violation of Equation (1.3). A model of context effects is a model that is allowed to break IIA in a controlled way.

1.5 THE CONTEXT-DEPENDENT RANDOM UTILITY MODEL

Seshadri, Peysakhovich and Ugander propose the simplest controlled way of breaking IIA (Seshadri et al., 2019).

1.5.1 The Idea: Every Item Plays Two Roles

In the CDM every item has two roles at once. It is a **target**, the item being evaluated for selection, and it is a **context player**, an item whose presence shifts how attractive the others look. In the streaming example Premium is a context player for Standard: its presence boosts Standard. The strength of that influence is a single number u_{xz}, the effect of the presence of z on the utility of x.

1.5.2 From the Universal Logit to the CDM

Start from the universal logit in Equation (1.2). Write $v(x) \equiv v(x \mid \{x\})$ for the utility of x when it stands alone, and approximate the utility in a larger set by a first-order expansion around that point, adding up the effect of each other item taken separately:

$$v(x \mid C) \;\approx\; v(x) + \sum_{z \in C \setminus \{x\}} u_{xz}, \qquad u_{xz} = v\big(x \mid \{x, z\}\big) - v(x). \tag{1.4}$$

So u_{xz} is how much the utility of x changes when z is added next to it, and the effects of different items are assumed to add. Substituting into Equation (1.2) gives

the CDM,

$$\Pr(x, C) = \frac{\exp\left(v(x) + \sum_{z \in C \setminus \{x\}} u_{xz}\right)}{\sum_{y \in C} \exp\left(v(y) + \sum_{z \in C \setminus \{y\}} u_{yz}\right)}. \tag{1.5}$$

When all $u_{xz} = 0$ the context disappears and the model is the MNL.

1.5.3 Low-Rank CDM and Identifiability

With n items the CDM has $O(n^2)$ parameters. The low-rank version factorises the interaction matrix,

$$u_{xz} = \langle \alpha_x, \beta_z \rangle = \alpha_x^\top \beta_z, \tag{1.6}$$

with a target embedding $\alpha_x \in \mathbb{R}^r$ and a context embedding $\beta_z \in \mathbb{R}^r$ for each item, reducing the count to $O(nr)$. The authors also show that the full CDM cannot be identified from choice sets of a single size, but can be identified when the data contain choice sets of sizes $3, \ldots, n-1$. A practical lesson is that the data must show the model enough variety in the sets it sees.

1.5.4 What the CDM Cannot Do

The CDM learns a separate u_{xz} for every pair of specific items. This has two drawbacks. First, it cannot say anything about an item it has not seen, since the parameters are tied to item identities. Second, it is opaque: it tells us that z affects x but not *through which attribute*. If we want effects that transfer to new items, and that can be read off in terms of features like price and star rating, we need the effects to live on features rather than identities. This is the step taken next.

1.6 THE LINEAR CONTEXT LOGIT

Tomlinson and Benson keep the idea of a context effect but move it from items to features (Tomlinson and Benson, 2021). The question changes from "does hotel A affect hotel B?" to "does the *average price* of the hotels shown change how much I care about star rating?"

1.6.1 The Model

Let the mean feature vector of the choice set be

$$x_C = \frac{1}{|C|} \sum_{j \in C} x_j \in \mathbb{R}^d. \tag{1.7}$$

The LCL replaces the fixed preference vector θ of the conditional logit by a set-dependent one, $\theta + Ax_C$:

$$\Pr(i, C) = \frac{\exp\left([\theta + Ax_C]^\top x_i\right)}{\sum_{j \in C} \exp\left([\theta + Ax_C]^\top x_j\right)}. \tag{1.8}$$

The matrix $A \in \mathbb{R}^{d \times d}$ is the **context-effect matrix**. The entry A_{pq} says how much the weight placed on feature p changes when the set's mean of feature q goes up (column causes, row is affected). When $A = 0$ the model reduces to the conditional logit.

1.6.2 Why LCL?

- **Interpretable.** All context effects sit in one $d \times d$ matrix whose entries have a feature-level meaning.
- **Generalises to new items.** The model is defined on features, so a model trained on one city's hotels can be applied to another's.
- **Convex estimation.** The utility is linear in (θ, A), so the negative log-likelihood is convex and gradient descent finds the global optimum.

Tomlinson and Benson also describe a decomposed variant (DLCL) that mixes d components, one per feature, to capture different kinds of users. We use the single-component LCL throughout.

1.7 CDM AND LCL SIDE BY SIDE

The two models describe context through different mechanisms. The CDM adds up *pairwise* effects of each other item on the target, and the LCL lets the *aggregate* of the set tilt a shared preference vector. They are related. If we tie the CDM's pairwise terms to features, say $u_{ij} = x_i^\top C_{\text{int}}\, x_j$ with $C_{\text{int}} \in \mathbb{R}^{d\times d}$ (our own feature-based variant, used to generate CDM data), the CDM's context term becomes

$$\sum_{j\in C\setminus\{i\}} x_i^\top C_{\text{int}}\, x_j \;=\; x_i^\top C_{\text{int}} \sum_{j\in C\setminus\{i\}} x_j, \tag{1.9}$$

which has the same "item × set summary" bilinear shape as the LCL term $(Ax_C)^\top x_i = x_i^\top A\, x_C$. The differences are that the CDM sums over the *other* items while the LCL averages over *all* of them, and that the LCL carries a separate context-free preference θ.

Two features of both models matter for the rest of this thesis. They are explicit: all context effects live in a small matrix we can write down. And they are generative: we can choose θ and A, simulate choices from them, and then know exactly what a learner *should* find. Real data never offer that.

1.8 SUMMARY

- People's choices depend on the set of alternatives (Tversky, Simonson and others), and context effects have been found in real booking data.
- Learning-to-rank systems score each item independently, which amounts to the MNL assumption and cannot represent such effects.
- The CDM puts the effect in pairwise interactions between items; the LCL puts it in a feature-level matrix A driven by the set's mean features.
- Both are interpretable models of context, but both require us to decide the form of the interaction in advance.

That last point is where we go next. Suppose we do not want to commit to a mechanism, and instead use a flexible model that is simply allowed to look at every other item before scoring one. One such model, already standard in ranking, is **self-attention**. Chapter 2 asks what it is, why it should be able to capture context effects, and what has not yet been checked about whether it actually does.

CHAPTER 2

SELF-ATTENTION AS A MODEL OF CONTEXT

Chapter 1 ended with two interpretable models of context, the CDM and the LCL, each of which fixes the *form* of the interaction in advance. This chapter turns to a model that does not. Self-attention lets every item look at every other item before it is scored and leaves the network to work out what, if anything, to do with what it sees. We explain what self-attention is and how it evaluates a choice set, show with a worked example how it can reproduce an LCL, review two earlier works that used set-aware neural models for ranking and choice, and then say what those works did *not* check. That gap is the motivation for the experiments in the rest of the thesis.

2.1 THE IDEA BEHIND SELF-ATTENTION

Think of the hotel list again. A context-free ranker reads one hotel at a time, as if each were shown alone. Self-attention adds one step before the score is produced. Each hotel *looks around* at the other hotels in the list, decides which of them matter for it, and collects a summary of what it sees. Its score is then computed from its own features together with that summary.

To make "looks around" concrete, each item plays three parts: it forms a **query** ("what do I want to know about the others?"), it publishes a **key** ("this is what I am like") and it carries a **value** ("this is the information I pass on"). An item compares its query with every key, turns the comparisons into weights that sum to one, and takes the weighted average of the values. That average is the context summary.

2.2 THE ARCHITECTURE

2.2.1 Self-Attention for a Choice Set

Let a choice set $C = \{1, \dots, m\}$ have item features $x_1, \dots, x_m \in \mathbb{R}^d$, and let $h_i \in \mathbb{R}^{d_h}$ be the embedding of item i (for example $h_i = W_E x_i$). Three learned matrices $W_Q, W_K \in \mathbb{R}^{d_k \times d_h}$ and $W_V \in \mathbb{R}^{d_v \times d_h}$ produce queries, keys and values,

$$q_i = W_Q h_i, \qquad k_j = W_K h_j, \qquad v_j = W_V h_j. \tag{2.1}$$

The attention weight that item i gives to item j is a softmax over the set of scaled query–key products,

$$a_{ij} = \frac{\exp\left(q_i^\top k_j / \sqrt{d_k}\right)}{\sum_{l=1}^{m} \exp\left(q_i^\top k_l / \sqrt{d_k}\right)}, \tag{2.2}$$

and the output for item i is the weighted average of the values,

$$o_i = \sum_{j=1}^{m} a_{ij}\, v_j. \tag{2.3}$$

In matrix form this is the familiar $\text{Attention}(Q, K, V) = \text{softmax}\left(QK^\top / \sqrt{d_k}\right)V$ (Vaswani et al., 2017).

2.2.2 The Encoder Block and the Readout

A full **encoder block** wraps this operation with several heads in parallel (each with its own W_Q, W_K, W_V, so that different heads can look for different things), a residual connection, layer normalisation and a position-wise feed-forward network (FFN):

$$\tilde{h}_i = \text{LayerNorm}\left(h_i + \text{MultiHead}(h_1, \dots, h_m)_i\right), \tag{2.4}$$

$$h'_i = \text{LayerNorm}\left(\tilde{h}_i + \text{FFN}(\tilde{h}_i)\right). \tag{2.5}$$

Blocks can be stacked, so that the representations produced by one block are what the next block's items look at. For choice modelling, a readout maps each final representation to one number $s_i = f(h'_i)$, and the choice probabilities are a softmax

over the set,

$$\Pr(i, C) = \frac{\exp(s_i)}{\sum_{j \in C} \exp(s_j)}. \tag{2.6}$$

This is the same form as the MNL in Equation (1.1), with one difference: the score s_i now depends on the whole set C rather than on x_i alone.

2.2.3 Properties Relevant to Choice

- **Permutation equivariance.** Without positional encodings, reordering the items reorders the outputs in the same way and changes nothing else. A model of choice should not care in which order the items are listed.
- **Variable set size.** Padding slots can be masked out of Equation (2.2), so a single model handles sets of any size.
- **Global interaction.** Every item can attend to every other in a single layer, so the score of each can depend on the whole set.

2.3 HOW SELF-ATTENTION EVALUATES A CHOICE SET:

To see what "evaluate" means, return to the two-hotel example of Section 1.6, with $\theta = [-1, 2]$, $A = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}$ and $C = \{[1, 3], [3, 5]\}$. Take the simplest attention layer: all attention scores equal, so that $a_{ij} = 1/m$, and $W_V = I$. Then the context summary of every item is just the set mean,

$$o_i = \frac{1}{m} \sum_{j=1}^{m} W_V x_j = W_V x_C = [2,\ 4]. \tag{2.7}$$

Now let the readout combine this summary with the item's own features by a product, $s_i = \theta^\top x_i + x_i^\top B\, o_i$, and set $B = A$. For the first hotel $\theta^\top x_1 = 5$ and $x_1^\top A\, o_1 = [1, 3] \cdot [0, 2] = 6$, giving $s_1 = 11$; for the second, $7 + 10 = 17$. These are exactly the LCL utilities, so the choice probabilities are those of Equation (1.8), 0.0025 and 0.9975. In other words, a self-attention network *can* compute the LCL, provided its weights have the right values.

2.4 CAN SELF-ATTENTION EXPRESS CONTEXT EFFECTS?

The example above was a special case. In general the connection to the models of Chapter 1 has three parts. Take $h_i = x_i$ for simplicity.

2.4.1 Attention Logits as CDM-Style Pairwise Interactions

The un-normalised attention score between items i and j is

$$q_i^\top k_j = (W_Q x_i)^\top (W_K x_j) = x_i^\top \underbrace{W_Q^\top W_K}_{M} x_j, \tag{2.8}$$

which has the bilinear form $x_i^\top C_{\text{int}} x_j$ of the feature-based CDM in Equation (1.9), with an interaction matrix M of rank at most d_k. The factorisation also mirrors the low-rank CDM of Equation (1.6): $W_Q x_i$ plays the part of the target embedding α_x and $W_K x_j$ that of the context embedding β_z. The difference lies in how the scores are used. The CDM *adds* them to the utility. Attention *normalises* them into weights for mixing value vectors.

2.4.2 Attention Can Compute the LCL Set Summary

With uniform attention, as in the example, $o_i = W_V x_C$ is a linear image of the set mean that drives the LCL. If the readout combines it multiplicatively with the item, $s_i = \theta^\top x_i + x_i^\top B\, o_i$, the result is the LCL utility with $A = B\, W_V$.

2.4.3 Why a Multiplicative Interaction Is Needed

The product in the readout is essential. If the readout were merely linear, $s_i = w^\top (x_i + o_i) = w^\top x_i + w^\top W_V x_C$, the context term would be the *same for every item in the set* and would cancel in the softmax, so no context effect would survive. (We ignore layer normalisation, which is itself non-linear.) The item–context interaction has to come from somewhere: query-dependent attention weights, the feed-forward network, a second attention layer or a non-linear readout. With query-dependent weights o_i differs from item to item and a single layer can produce an effect, but it

is then a softmax-weighted function of the set, not exactly the linear-in-x_C form of the LCL. This is one reason why the depth and width of the network are variables in our experiments.

2.4.4 Expressivity Is Not Identification

The constructions above show that self-attention *can* represent CDM- and LCL-type context effects. They do not show that a self-attention model trained by maximum likelihood *will* learn them. Two obstacles remain.

- **Many networks, same predictions.** A deep network has many parameter settings that give nearly identical choice probabilities on the training data. Nothing in the objective prefers the one that implements θ and A faithfully over one that fits the observed sets in another way.
- **Attention weights need not carry the effect.** Even when the model's *behaviour* reflects the true context effect, it may be spread across the values, the FFN and the readout, not sitting in the weights a_{ij}. Reading context effects off attention weights and measuring them from the model's outputs can therefore give different answers.

2.5 PRIOR WORK: SET-AWARE NEURAL MODELS FOR RANKING AND CHOICE

Two earlier works put this idea into practice. For each we state what was done and what it leaves open.

2.5.1 Context-Aware Learning to Rank with Self-Attention

Pobrotyn et al. apply a Transformer encoder to learning to rank (Pobrotyn et al., 2020). Each document in a query's candidate list is embedded from its features; sinusoidal positional encodings based on its initial rank are added; the list passes jointly through stacked self-attention blocks so that every document's representation

depends on all the others; and a scalar relevance score is read out. The model is trained against a differentiable approximation of NDCG and improves ranking metrics over context-free baselines such as LambdaMART. The authors attribute the gains to the model's ability to use the context of the whole list.

What it leaves open:

- **Context is inferred, not measured.** The evidence that the model uses context is a better aggregate ranking metric. With no known ground truth it cannot be said *what* context effect, if any, the model has learned.
- **Confounding signals.** When an item's position is visible to the model, an improvement can come from that signal, not from interactions between items. In our own earlier replication on Expedia hotel-search data, a position-only ablation outperformed the full attention model (0.3922 versus 0.3848 NDCG@10): a concrete case of gains credited to context that were not clearly coming from it.
- **No structural analysis.** The learned interactions are never compared with an interpretable choice model, so there is no check that they correspond to any known context effect.

2.5.2 FETA and FATE: Learning Context-Dependent Choice Functions

Pfannschmidt et al. approach the problem from the choice side (Pfannschmidt et al., 2022). They formalise context-dependent utility functions $U(x, C)$ and propose two neural architectures that are permutation invariant in C and handle sets of any size.

- **FETA** (First Evaluate, Then Aggregate) writes utility as a context-free term plus the average of learned pairwise terms,

$$U(x, C) = U_0(x) + \frac{1}{|C| - 1} \sum_{z \in C \setminus \{x\}} U_1(x, z), \tag{2.9}$$

  a neural, first-order analogue of the CDM.

- **FATE** (First Aggregate, Then Evaluate) embeds each item with a network ϕ, averages the embeddings into a set representation, and evaluates each item against it,

$$U(x, C) = U\Big(x,\ \tfrac{1}{|C|}\sum_{z \in C} \phi(z)\Big). \tag{2.10}$$

  With ϕ the identity and U linear-plus-bilinear, FATE reduces to the LCL, so it is a neural generalisation of it. The authors prove a universal representability result for FATE.

Self-attention sits between the two: like FETA it computes pairwise scores (Equation (2.8)), like FATE it aggregates over the set (Equation (2.3)), and unlike either it *learns* how to weight that aggregation.

What it leaves open:

- **Judged on prediction.** The architectures are evaluated by how well they predict choices, and good prediction does not show that the context effects that generated the data were learned.
- **Representability is not recoverability.** Universal representability says a suitable network *exists*. It says nothing about whether training on finite data finds it, or how much data that takes.
- **Restricted forms.** FETA is limited to first-order pairwise terms and FATE to a uniform average. Neither tests whether a learned, flexible aggregation such as self-attention helps or hurts structural recovery.
- **No comparison with ground truth.** The learned functions U_1 and ϕ are never mapped back to interpretable quantities such as a context-effect matrix A, so recovery is never quantified.

2.6 MOTIVATION: TESTING RECOVERY, NOT JUST ACCURACY

The two lines of work have a common thread. Both show that models which look at the whole set *predict* well. Neither asks whether the model has found the *reason*.

In the first work the improvement is simply credited to context; in the second, representability is proved but not tested against a planted effect.

Meanwhile Chapter 1 gave us exactly what is needed for such a test: the LCL, whose context-effect matrix A and preference vector θ we can choose, simulate from, and compare with what a trained network has learned. This thesis therefore asks the following questions.

- **RQ1.** When a self-attention model reaches near-oracle predictive accuracy on LCL data, does it also recover the underlying A and θ?
- **RQ2.** What governs how well the structure is recovered: the amount of training data, the rank and sparsity of A, the number of item features, or the depth of the network?
- **RQ3.** Does recovery matter in practice? Does it predict how well the model behaves on shifted choice sets that accuracy on in-distribution data cannot distinguish?

Our contributions are the following.

- A **framing** of context-effect learning as a recovery problem, measured separately from prediction.
- A **controlled evaluation methodology**: synthetic LCL data with planted, rank- and sparsity-controlled context effects, distribution-shifted evaluation sets, and a behavioural probe that reads the implied A and θ out of any trained scoring model, validated on the generator itself.
- A **systematic empirical study** sweeping training-set size, rank and sparsity of A, feature dimension, depth, random initialisation and optimisation settings over many random seeds. At the reference setting ($d = 16$, rank 5, 285,000 sessions), one-layer self-attention recovers A with cosine 0.990 and relative error 0.150 ± 0.040, against 0.092 ± 0.010 for the oracle that knows the LCL form. It is within noise of a mean-pool network (0.142 ± 0.016) and clearly

better than a flat MLP of the same size (0.241 ± 0.035). Recovery is governed by the amount of data, the rank of A and the number of features, and not by depth, sparsity, learning rate or weight decay (Chapter 4).

- An **interpretability observation**: reading context effects directly off attention weights and measuring them behaviourally by probing give different answers.

2.7 SUMMARY

- Self-attention lets each item's score depend on the whole set. Its query–key logits have the bilinear form of CDM-style interactions, and with uniform weights its output is a linear image of the LCL set mean.
- Representing an LCL effect additionally needs a multiplicative item–context interaction, supplied by depth, query-dependent weights or non-linearity.
- Prior work shows that set-aware models predict well, but does not check whether they recover the mechanism.

What remains is to do that check. Chapter 3 describes how we generate LCL data with a known A and θ, which models we train on it, and how recovery is measured; Chapter 4 reports what we found.

CHAPTER 3

EXPERIMENTAL DESIGN: TESTING RECOVERY ON LCL DATA

Chapter 2 ended with one open question: when a self-attention model predicts context-dependent choices well, has it really learned the context effect behind them, or does it only predict well? To answer this we need data where the true effect is known. This chapter explains how we make such data from the LCL of Section 1.6, which models we train on it, how we train and sweep them, and how we measure prediction and recovery.

3.1 OVERVIEW OF THE EXPERIMENT

The experiment is a loop: (1) **plant** a preference vector θ and a context matrix A; (2) **simulate** choices from the LCL; (3) **train** models, including self-attention, on the choices; (4) **read out** the $\hat{A}$ and $\hat{\theta}$ each trained model uses; (5) **compare** them with the planted A and θ and with the ordinary prediction quality. The planted θ and A are used only in steps 2 and 5; training and model selection never see them. We simulate because real data have no true choice probabilities and no true A, so only prediction could be checked. Simulation lets us ask two separate questions: how well does a model *predict*, and has it *learned the rule* that made the choices?

3.2 GENERATING THE DATA

The data come from `lcl_core.py`. A session shows a slate S of five items; item i has a feature vector $x_i \in \mathbb{R}^d$, and

$$u_i = \theta^\top x_i + x_i^\top A\, y_S, \qquad y_S = \frac{1}{5}\sum_{j\in S} x_j, \qquad P(i \mid S) = \frac{e^{u_i}}{\sum_{j\in S} e^{u_j}}. \tag{3.1}$$

The slate mean y_S includes item i itself. With $A = 0$ this is a plain multinomial logit (MNL) with no context effect.

3.2.1 Ground Truth and Reference Setting

For each seed we draw $\theta \sim \mathcal{N}(0, I_d)$ scaled to $\|\theta\|_2 = 1$, and $A = UV^\top$ with $U, V \in \mathbb{R}^{d\times r}$ standard normal, so $\operatorname{rank}(A) = r$, scaled to $\|A\|_F = 1$. The matrix A is full ($d \times d$, not symmetric, diagonal included). In the sparsity sweep some entries of A are first set to zero. Both norms are 1, so the strength of the planted effect is the same in all seeds and a change in recovery comes from the swept factor only.

Reference setting. Unless a sweep changes it: $d = 16$ features, rank $r = 5$ and $n = 285{,}000$ training sessions. This keeps the sessions per entry of A the same as in our earlier pilot ($d = 6$, 40,000 sessions; $40{,}000 \times (16/6)^2 \approx 285{,}000$), but A now has 256 entries instead of 36. Figure A.1 (Appendix A) shows the planted truth of the first dataset (seed 101).

3.2.2 Simulated Choices and Reference Losses

Every feature of every item is drawn independently from $\mathcal{N}(0, 1)$, with five items per slate in random order. For each session we compute the exact probabilities of Equation (3.1) and draw one choice. For seed 101 each slate position gets 19.8% to 20.1% of the choices, and the base part $\theta^\top x_i$ has standard deviation 1.00 against 0.44 for the context part $x_i^\top A y_S$. Switching the context part off changes the most likely item in 23% of the sessions (median change 11 percentage points of probability). In half of the sessions the true top item has probability between 0.37 and 0.57, so choices are far from deterministic.

Loss. Every model is trained with the negative log-likelihood (NLL) of the chosen item. This is the same as the cross-entropy loss with the chosen item as the label. Four numbers (seed 101) are used again and again in Chapter 4:

- $\log 5 = 1.609$: the loss of guessing $1/5$ for each item. Not computed from data.
- 1.265: the loss of the *true* model, i.e. the mean entropy of the true probabilities P. No model can beat this on average.
- 1.326: the cross-entropy of P against the probabilities Q of the model with the true θ and the context switched off ($A = 0$). This is a formula; nothing is trained.
- $0.061 = 1.326 - 1.265$: the *value of the context*. A model 0.061 above the true loss has learned no context; a model at 0 has learned all of it. All model gaps are read against this number.

The true probabilities P are stored for every split, but *training never uses them*: the models see only the chosen item. P is used only for grading (true loss, KL, agreement with the true top item). Recovery of A and θ is graded against the planted A and θ.

3.2.3 Splits, Seeds and Worlds

A *dataset* is one random world: its seed fixes θ and A, and the seed plus one fixes the features and choices. It has four parts: a **training set** of n sessions; a **validation set** of 5,000 sessions, used only for early stopping and tuning; an **in-distribution evaluation set** of 5,000 new sessions (the reported test numbers); and a **shifted evaluation set**: 100,000 random slates are drawn and the 5,000 with the largest $\|y_S\|_2$ (top 5%) are kept, so the context term is large and such slates are rare in training.

For false-positive checks each seed is run in two *worlds*: the *context* world with the A above, and the *no-effect* world with $A = 0$ (same θ); a sound read-out must

give $\hat{A} \approx 0$ in the second. Hyperparameters were tuned on seeds 1 and 2; reported experiments use other seeds (101 to 130 for the headline experiment, 201 to 210 for initialisation), so no setting was chosen on the data it is reported on.

3.3 MODELS

Every model gives one score s_i to each item, and the choice probabilities are the softmax of the scores over the slate, as in the LCL. Table 3.1 lists the models and their sizes at $d = 16$.

Table 3.1: Models compared in Chapter 4, at $d = 16$. The three models marked $*$ are matched at about 9,200 parameters. Oracle and small attention models: Table 3.2.

Model	Structure	Parameters
Self-attention, 1 layer $*$	width 32, 2 heads	9,185
2 / 4 / 6 layers	grows with depth	17,729 / 34,817 / 51,905
Mean-pool, 2 layers $*$	width 80	9,201
4 / 6 layers	width 80, grows with depth	22,161 / 35,121
Flat MLP, 2 layers $*$	width 62	9,243
4 / 6 layers	narrower (width 42 / 35)	9,035 / 9,315
Wide flat MLP, 4 / 6 layers	width 62, grows with depth	17,055 / 24,867

3.3.1 The Self-Attention Model

The self-attention model (`sa_L`L) is a standard transformer encoder on the five items (Figure 3.1). It has **no positional encoding** and **no masking**: every item attends to every item of its slate, itself included. So it is permutation-equivariant (reordering the slate only reorders the scores). With $L = 1$ it has 9,185 parameters.

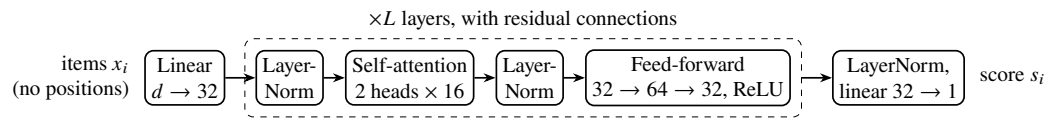


Figure 3.1: The self-attention model. The same weights are used for every item; items interact only through attention. The five scores go to a softmax over the slate. No dropout.

No dropout. Dropout is 0 in our self-attention model, although standard transformers use it. Our reasons: dropout fights overfitting on small data, but here the training

set is large (285,000 sessions for about 9,000 weights), the gradient is exact (full batch), and early stopping already controls overfitting; dropout also adds random noise to the scores, which may blur the rule we want to read out. We did *not* test this, and dropout was not treated as a tuning knob. It is a limitation (Section 3.7).

3.3.2 Mean-Pool and Flat Networks

Why two networks. We use two networks without attention as reference points, and they answer two different questions. The mean-pool network is *given* the slate mean y_S, which is exactly the quantity the LCL context term depends on, so it shows what a plain MLP can reach when the set summary is supplied. The flat MLP gets only the five items written one after the other, so it shows what happens with no set structure at all. Self-attention sits between them: it is not given y_S, but it is built to look across the slate. Comparing with the flat MLP asks whether set structure matters at all, and comparing with mean-pool asks whether self-attention *finds* the slate summary by itself. At 2 layers the three models have about the same size (Table 3.1), so a difference comes from structure and not from the number of parameters. With more depth a flat MLP could in principle learn the slate average by itself, so we also trained deeper versions; the wide flat MLP below keeps its width as depth grows, like mean-pool, and is the fair depth comparison (Section 4.3.5).

Mean-pool (`mp_L`L): for each item we form $[x_i,\ y_S]$ (32 numbers) and pass it through the same MLP ($32 \to 80 \to 80 \to 1$ at $L = 2$). It is permutation-equivariant and is *given* the slate mean, so it only has to learn how x_i and y_S interact. **Flat MLP** (`flat_L`L): the five items are written one after the other, $[x_1, \ldots, x_5] \in \mathbb{R}^{80}$, and an MLP ($80 \to 62 \to 62 \to 5$ at $L = 2$) gives the five scores. It does not know that the slate is a set and must learn this from data. It is not permutation-equivariant, so its probe and Metric C readings (Section 3.6) are taken after a random reordering of each slate. The *wide flat MLP* (`flatw_L`L) keeps the width of the 2-layer version at every depth.

Fair size. The widths of mean-pool and flat MLP are the integer widths whose parameter count is closest to that of 1-layer self-attention (`lcl_core.base_width`),

so the three main models differ in structure, not size. **In the depth sweep not every family stays size-matched** (Table 3.1): the flat MLP is re-matched at every depth (it becomes narrower), while self-attention, mean-pool and the wide flat MLP keep their width and grow. The wide flat MLP tests the flat network when it is bigger.

3.3.3 Oracle and Two Small Attention Models

Three small models contain the LCL form and show what is *achievable*. They are described only here and are not used in the comparisons of Chapter 4.

- **Oracle LCL** (`oracle`): Equation (3.1) with free θ and A, fitted by the same training code. Its error is the floor set by the finite data.
- **Bilinear attention** (`bil_h`h): $s_i = w^\top x_i + \frac{1}{5}\sum_j (W_q x_i)^\top (W_k x_j)$ with $W_q, W_k \in \mathbb{R}^{h\times d}$. This *is* the LCL, with $A = W_q^\top W_k$ of rank at most h.
- **Softmax attention head** (`sov_h`h): $s_i = w^\top x_i + (W_o x_i)^\top \sum_j \alpha_{ij} W_v x_j$, with α_{ij} a softmax over $(W_q x_i)^\top W_k x_j$. If all weights are equal ($\alpha_{ij} = 1/5$) it gives the LCL with $A = W_o^\top W_v$, so it *can represent* the LCL. In training the weights are free to become unequal.

Table 3.2 shows them on the 30 headline datasets. Both are at or very close to the oracle in prediction and in recovery, so the data floor for the error in $\hat{A}$ is about 0.09 for a model with the right form.

Table 3.2: Oracle, bilinear attention and softmax head ($h = 16$): mean over the 30 headline datasets ($d = 16$, rank 5, $n = 285{,}000$). Errors are relative (Section 3.6); excess NLL is the test loss minus the oracle's on the same dataset, in 10^{-3} nats.

Model	Parameters	Error in A	Error in θ	Excess NLL	KL (10^{-3})
Oracle LCL	272	0.092	0.020	0.00	0.69
Bilinear, $h = 16$	528	0.088	0.022	−0.05	0.68
Softmax head, $h = 16$	1,040	0.091	0.025	0.28	1.11

Head size and rank. Do not mix two ranks: the *true rank* r of the planted A (the data) and the *head size* h (the model). For a head of size h, the best possible cosine with A is the Eckart–Young value $\sqrt{\sum_{i\le h}\sigma_i^2 / \sum_i \sigma_i^2}$ (σ_i the singular values of A).

We trained both heads on a grid of $r, h \in \{1, 2, 4, 8, 16\}$ (10 datasets per cell); in every cell the achieved cosine is within 0.006 of this best value. So a head that is too small falls short only because of its size, not because training failed.

3.4 TRAINING AND HYPERPARAMETERS

All models use the same training code (`lcl_core.train_model`). The loss is the NLL of Section 3.2.2. The other fixed choices are below; we made them once and did not sweep them (Section 3.7).

- **Optimiser:** AdamW with a constant learning rate (no schedule, warm-up or clipping); a common default.
- **Batch:** full batch, in chunks of 50,000 that add up to the exact gradient, so one epoch is one update and there is no batch noise.
- **Stopping:** at most 500 epochs; stop after 60 epochs without a new best validation loss and keep the best epoch. Same budget for all models, and A and θ are never used.
- **Initialisation:** PyTorch default for self-attention, mean-pool and flat MLPs; normal with std. 0.1 for the oracle, bilinear and softmax head.

Tuning. For each model we trained all six pairs of learning rate $\{0.003, 0.01, 0.03\}$ and weight decay $\{0, 10^{-4}\}$ on two datasets at the reference setting and kept the pair with the lowest mean validation loss (the chosen pairs are in Appendix A). The choice matters little: the worst pair is at most 0.0004 nats above the best for the oracle, bilinear and softmax head, 0.0026 for self-attention and 0.0016 for mean-pool, small against the gap of about 0.02 nats between the flat MLP and self-attention. Several models chose 0.03, the top of the grid, so Section 4.3 checks directly that recovery does not depend on it. With only two tuning datasets, differences below about 0.001 nats are not meaningful.

Training budget. Of the 5,020 trainings in the main experiments, 64 (1.3%) were still improving at the 500-epoch limit: 38 self-attention models, 24 flat MLPs (19 with 6 layers) and 2 softmax heads. They are flagged in Chapter 4.

3.5 SWEEPS

Each sweep changes one factor and keeps the others at the reference setting; every seed makes a new θ, A, item features and choices (Table 3.3; full list in `tasks.py`).

Table 3.3: Experiments. SA: self-attention; MP: mean-pool; flat: flat MLP. "Both worlds": with and without a context effect.

Experiment	Values	Models	Seeds
E1 reference	reference setting; both worlds	all models	30
E2 data size	$n \in \{18, 36, 71, 142, 285, 570, 1140\}$ thousand; both worlds	SA, MP, flat, oracle	15
E3 rank of A	true rank r and head size h, each in $\{1, 2, 4, 8, 16\}$	all	10
E4 sparsity	zeros in A: 0 to 0.8 (rank 16); $n \in \{71\text{k}, 285\text{k}\}$	SA, MP, flat, oracle	15
E4 features	$d \in \{6, 12, 16, 24, 32\}$, rank about $d/3$; data grows as d^2 or is fixed at 40k	SA, MP, flat, oracle	10
E5 depth	SA 1, 2, 4, 6 layers; MP, flat 2, 4, 6; $n \in \{71\text{k}, 285\text{k}\}$	SA, MP, flat, wide flat, oracle	15
E6 initialisation	same data, 20 initialisations	SA, MP, oracle	10×20
E7 optimisation	all 6 learning-rate and weight-decay pairs; half and double width	SA, MP, flat	10

3.6 EVALUATION METRICS AND THE PROBE

We report prediction and recovery side by side.

Prediction. **Excess NLL** is the test NLL minus the oracle's on the *same* dataset; this removes the dataset-to-dataset noise, which is larger than the gaps between good models. **KL to the truth** is $\mathrm{KL}(P \,\|\, \hat{P})$ averaged over test sessions; it uses the true P, so it has no sampling noise, and 0 is perfect. We use three accuracies: *accuracy* (the model's top item is the chosen item), *expected accuracy* (the same, averaged over the randomness of the choice using P, so less noisy) and *agreement with the truth* (the model's top item is the true model's top item).

The behavioural probe. The probe reads $\hat{A}$ and $\hat{\theta}$ from the *scores* of a trained model and never looks at its weights, so it works the same for every architecture.

1. Pick two items x_a, x_b and keep them fixed in two slots. Fill the other three slots with fresh random items, 500 times. If the model follows the LCL, the score gap is

$$g = s_a - s_b = \theta^\top \delta + \delta^\top A\, y_S, \qquad \delta = x_a - x_b, \tag{3.2}$$

   a straight line in y_S with slope $\delta^\top A$. We use gaps because a softmax fixes only score differences.

2. **Centre first.** Subtract the pair's mean gap and mean y_S. This removes everything that is the same for all fillers, whatever its form, so a context-free model, even a strongly nonlinear one, gives $\hat{A} = 0$.

3. Repeat for $K = 10d = 160$ pairs: $160 \times 500 = 80{,}000$ scored slates per model. After centring, each pair gives 16 slope numbers ($\delta^\top A$), so one joint least squares solves $\hat{A}$ from $160 \times 16 = 2{,}560$ constraints for 256 unknowns.

4. With $\hat{A}$ fixed, the mean gap of each pair estimates $\delta^\top \theta$, and least squares over the pairs gives $\hat{\theta}$. Resampling the pairs (200 bootstrap samples, fewer for $d > 24$) gives confidence intervals.

Probe items and fillers come from the same $\mathcal{N}(0, I_d)$ as the training items, so the probe never makes a model extrapolate; all models get the same pairs. $\hat{A}$ **describes the model's behaviour** (the best LCL-shaped linear summary of it), **not its weights**; the fidelity below says how good the summary is.

Recovery measures. **Error in** A is $\|\hat{A} - A\|_F / \|A\|_F$ (0 is perfect, 1 is no better than $\hat{A} = 0$). The **cosine** between $\hat{A}$ and A tells whether the pattern is right, and the **scale ratio** $\|\hat{A}\|_F / \|A\|_F$ whether the strength is right. Error and cosine for θ are defined in the same way. **Fidelity** is the share of the variance of the centred gaps explained by $\delta^\top \hat{A} y_S$ (1 means exactly LCL-shaped). **Invariance** is

$\|\hat{A} - \hat{A}_{\text{shift}}\|_F / \|\hat{A}\|_F$, with $\hat{A}_{\text{shift}}$ probed on the 500 (of 5,000 per pair) slates with the largest $\|y_S\|$; 0 means one rule everywhere. **Metric C**: in 4,000 trials the same pair is placed in two independent random slates S, S', and the change of the model's gap, $g(S') - g(S)$, is regressed on the change predicted by the true A, $\delta^\top A(y_{S'} - y_S)$. A slope of 1 means the model reacts to the slate as much as the truth, below 1 it under-reacts; it uses the true A, so it is an independent check. The **false-positive floor** is $\|\hat{A}\|_F$ in the no-effect world (truth 0).

Checking the probe. We first ran the probe on four functions with a known answer, at $d = 6$ to 32. For the generator's own scores (V1) and a hand-built bilinear attention layer made from the SVD of A (V2), every entry of $\hat{A}$ and $\hat{\theta}$ is correct to at most 7.8×10^{-16}, the rounding level of double precision. For a context-free nonlinear function $\theta^\top x_i + \sum_k \sin x_{ik}$ (V0) it finds $\|\hat{A}\|$ between 4.0×10^{-31} and 2.3×10^{-30}, which is zero. For a saturating non-LCL effect $\theta^\top x_i + 0.5 \tanh(2\, x_i^\top A y_S)$ (V4), fidelity drops to 0.85–0.89 and the Metric C slope to 0.57–0.62. So the probe is exact when the answer is known, does not invent context, and flags non-LCL behaviour.

3.7 LIMITS AND REPRODUCIBILITY

We list what the design does not cover. (i) **Simulated LCL data only:** features are Gaussian and the true rule is exactly LCL; we say nothing about real choice data. (ii) **Slate size** is always 5. (iii) **Architecture:** the number of heads (2) was not varied, width only by the half and double width runs, depth was swept, and dropout was neither used nor tested. (iv) **Training:** optimiser, full batch, constant learning rate, patience (60) and epoch cap (500) were fixed; only learning rate, weight decay, width, depth and initialisation were varied. (v) **Tuning** used two datasets, and the depth sweep is not size-matched for all families (Section 3.3.2). (vi) **Probe settings** (500 slates per pair, $K = 10d$ pairs) were fixed, not swept.

Code is at `github.com/ajazhsn/Ajaz_Hussain_MTP-`: `lcl_core.py` (data, models, training, probe), `tasks.py` (every experiment and seed) and the notebook

behind every figure and number of Chapter 4. Runs used V100 GPUs and CPU nodes of the IITM DSAI WSAI cluster.

CHAPTER 4

RESULTS

Chapter 3 explained the data, the models and the metrics. This chapter gives the results, in the same order as Chapter 3: first how the models train, then the reference setting (Does a model that predicts well also recover A? This is RQ1), then the sweeps (What decides recovery? RQ2), and last the shifted slates (RQ3). Unless we say otherwise, numbers are mean ± standard deviation over datasets at the reference setting ($d = 16$, rank 5, $n = 285{,}000$). In the figures, error bars and bands are 95% confidence intervals of the mean.

4.1 TRAINING BEHAVIOUR

First we look at how the models train. Each of the four main models was trained once more on the first reference dataset with the protocol of Section 3.4, and the training and validation loss were saved after every epoch (Figure 4.1). The two reference lines are the loss of the true model on the same training and validation data; this is the level that a perfect model reaches.

The oracle settles exactly on the true model's loss, for both training and validation, as it should (it already has the right formula, so it only has to find 272 numbers). Self-attention and mean-pool come close to the true model's loss on both sets. Self-attention is the slowest: it is kept at epoch 211 (of 271 trained) and its curve is noisy, while mean-pool is kept at epoch 66 (of 126), the oracle at epoch 86 (of 146) and the flat MLP at epoch 100 (of 160). After its best epoch, the mean-pool validation loss slowly goes up while its training loss keeps going down, which is the usual sign of starting to overfit, and early stopping keeps the best epoch. The flat MLP behaves in a different way. Its training loss goes *below* the true model's

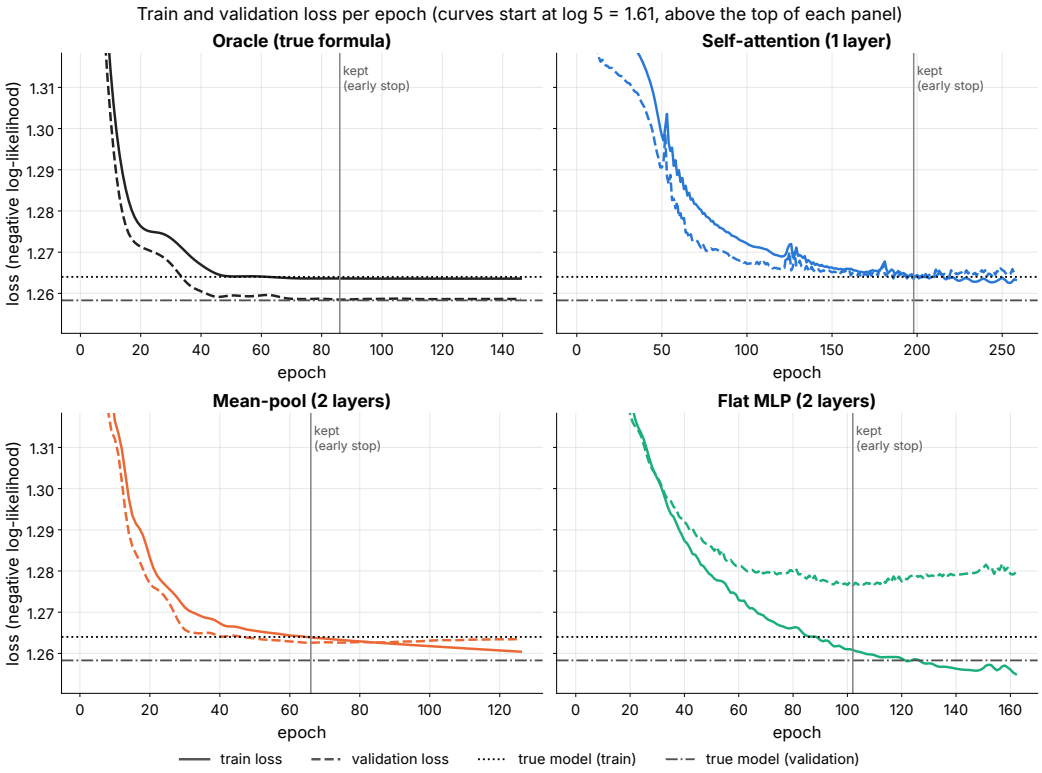


Figure 4.1: Training (solid) and validation (dashed) loss per epoch on the first reference dataset. Dotted and dash-dotted lines: loss of the true model on the training and validation data. Grey vertical line: the epoch kept by early stopping. All curves start at $\log 5 = 1.61$, above the top of each panel.

loss, but its validation loss stays about 0.02 nats above it. With the same number of parameters it finds it easier to memorise the noise in 285,000 sessions than to learn the context rule.

4.2 REFERENCE SETTING: ACCURACY VERSUS RECOVERY (RQ1)

Table 4.1 gives the main comparison over the 30 datasets of experiment E1, in the world with a context effect. The columns are the metrics of Section 3.6.

Table 4.1: Prediction and recovery at the reference setting (30 datasets, mean ± standard deviation). Excess NLL: test NLL minus the oracle's on the same dataset. KL: divergence of the model's choice probabilities from the true ones. Size: $\|\hat{A}\|/\|A\|$. Printed by notebook Section 4.2.

	Prediction ($\times 10^{-3}$)		Recovery of A			θ	LCL shape	
Model	Excess NLL	KL	Error	Cosine	Size	Rel. error	Fidelity	Metric C
Oracle LCL	0.00 ± 0.00	0.69 ± 0.29	0.092 ± 0.010	0.996 ± 0.001	1.01 ± 0.01	0.020 ± 0.011	1.00 ± 0.00	1.00 ± 0.01
Self-attention, 1 layer	4.78 ± 1.99	5.37 ± 1.02	0.150 ± 0.040	0.990 ± 0.005	0.93 ± 0.04	0.042 ± 0.010	0.91 ± 0.02	0.92 ± 0.04
Self-attention, 6 layers	4.50 ± 2.81	5.38 ± 2.13	0.163 ± 0.062	0.987 ± 0.012	0.93 ± 0.05	0.046 ± 0.017	0.92 ± 0.03	0.92 ± 0.06
Mean-pool, 2 layers	4.14 ± 1.43	4.72 ± 0.35	0.142 ± 0.016	0.991 ± 0.002	0.95 ± 0.02	0.032 ± 0.011	0.92 ± 0.00	0.94 ± 0.02
Flat MLP, 2 layers	22.06 ± 3.04	22.51 ± 1.17	0.241 ± 0.035	0.979 ± 0.007	0.85 ± 0.03	0.026 ± 0.004	0.59 ± 0.02	0.83 ± 0.03
Flat MLP, 4 layers	22.59 ± 3.07	23.10 ± 1.17	0.291 ± 0.034	0.969 ± 0.010	0.81 ± 0.03	0.034 ± 0.006	0.58 ± 0.02	0.79 ± 0.03
Flat MLP, 6 layers	23.87 ± 2.85	24.73 ± 1.73	0.332 ± 0.032	0.959 ± 0.010	0.79 ± 0.03	0.047 ± 0.010	0.55 ± 0.03	0.75 ± 0.04

4.2.1 Prediction

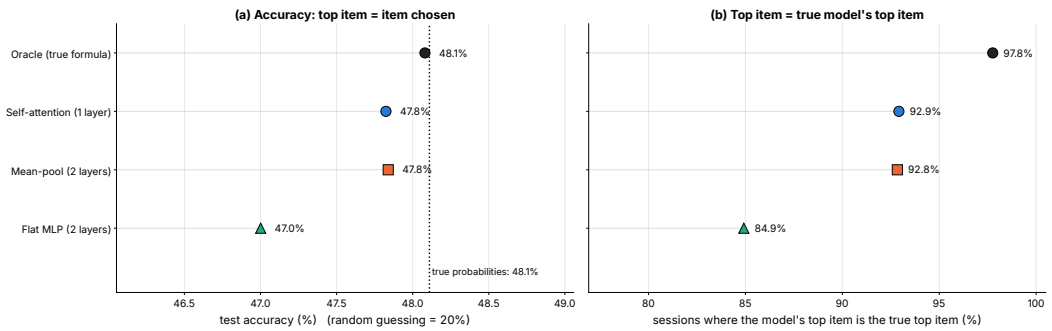


Figure 4.2: Accuracy on the first reference dataset. (a) How often the top-ranked item is the item chosen (dotted line: the true probabilities). (b) How often the top-ranked item is the true model's top item.

Plain accuracy is the first thing one would look at (Figure 4.2a), but it hardly separates the models. The choices are random, so even the true probabilities pick the chosen item only about 48% of the time, and all models lie within 1.5 percentage points of that (the true model: 48.4%; oracle 48.4%, mean-pool 48.1%,

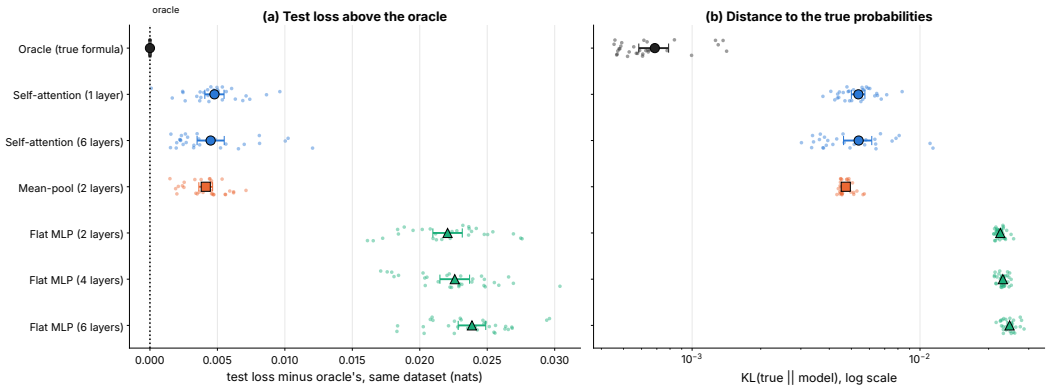


Figure 4.3: Prediction at the reference setting. Each small dot is one dataset; large markers are means with 95% confidence intervals. (a) Test loss minus the oracle's on the same dataset. (b) KL divergence to the true choice probabilities (log scale).

self-attention 48.0%, flat MLP 47.0%). Agreement with the true model's top item separates them better (Figure 4.2b). The oracle agrees with the true model's top item in 97.8% of the test sessions, mean-pool in 92.9%, one-layer self-attention in 92.5% and the flat MLP in 84.5%.

The loss-based measures (Figure 4.3) give three groups. The oracle is the reference. Self-attention and mean-pool are about 4–5 $\times 10^{-3}$ nats above it, and the flat MLPs are about 22–24 $\times 10^{-3}$ nats above it. The value of knowing the context is 0.061 nats (Section 3.2.2), so self-attention and mean-pool keep about 92–93% of this value and the flat MLPs only about 61–64%. More layers do not help the flat MLP; they make it slightly worse. One and six layers of self-attention cannot be told apart.

Two things are important when we compare the models. First, the raw test NLL changes from dataset to dataset with a standard deviation of about 0.009 nats, which is bigger than the gap between self-attention and mean-pool. So we always compare with the oracle on the same dataset (excess NLL) or use the KL to the true probabilities. Second, with KL, mean-pool (4.7×10^{-3}) is slightly but clearly better than one-layer self-attention (5.4×10^{-3}): their confidence intervals do not overlap. In excess NLL, where the spread is larger, they cannot be separated.

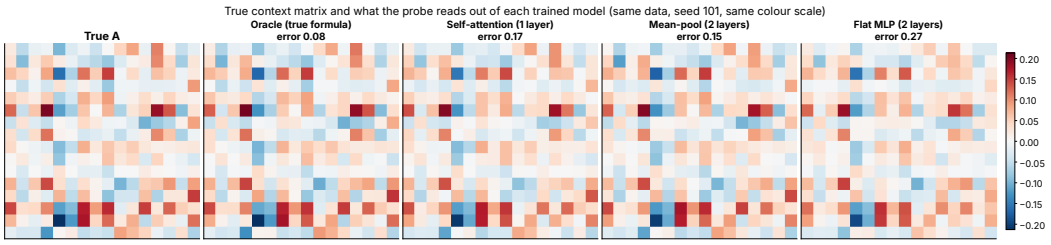


Figure 4.4: The true A of the first reference dataset and the $\hat{A}$ read from each trained model with the probe, on the same colour scale. The number under each title is $\|\hat{A} - A\|/\|A\|$ for this dataset.

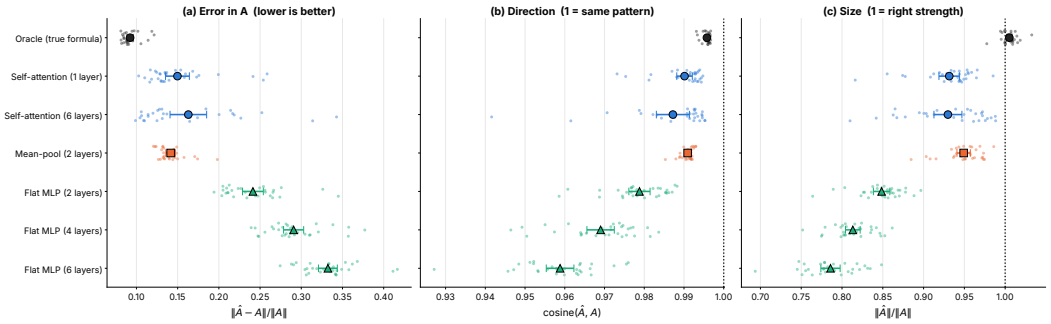


Figure 4.5: Recovery of A over 30 datasets. (a) Relative error. (b) Cosine between $\hat{A}$ and A. (c) Size $\|\hat{A}\|/\|A\|$.

4.2.2 Recovery of the context matrix

Figure 4.4 shows the first dataset: every model gives a matrix with the same main pattern as the true A. Over 30 datasets (Figure 4.5), the oracle has error 0.092 ± 0.010; this is the floor that comes from having only 285,000 sessions. Self-attention reaches 0.150 ± 0.040 and mean-pool 0.142 ± 0.016, about 1.5 to 1.6 times the floor, and their confidence intervals overlap. The flat MLP reaches 0.24 to 0.33, and it becomes worse with every layer.

The direction of $\hat{A}$ is correct for all models (cosine at least 0.96), but a cosine near 1 can hide real differences: 0.996 for the oracle and 0.990 for self-attention correspond to errors of 0.09 and 0.15. The free-form networks also *under-react*: self-attention recovers 93% of the true strength of A, mean-pool 95% and the flat MLP 79–85%. The error of self-attention also varies more between datasets than that of mean-pool (standard deviation 0.040 against 0.016); Section 4.3.6 shows that a good part of this comes from the random starting weights.

4.2.3 No false context effects

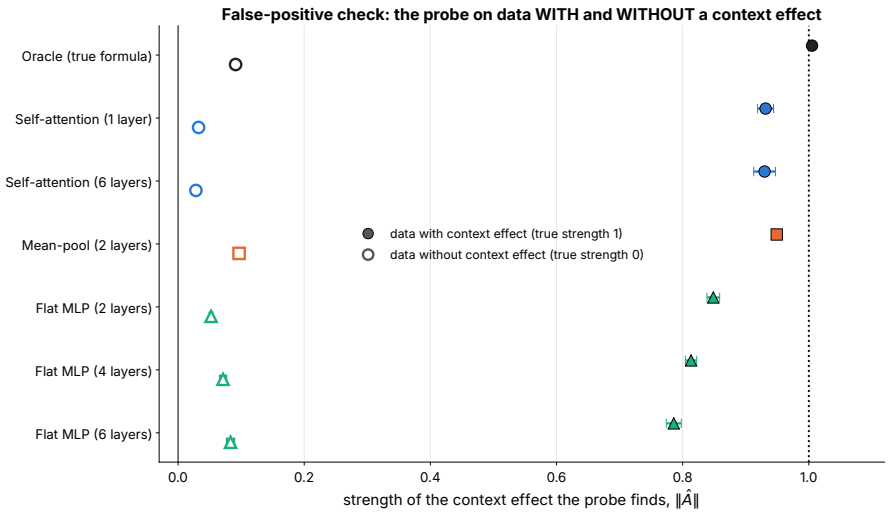


Figure 4.6: Strength of the context effect found by the probe, $\|\hat{A}\|$, for models trained on data with a context effect (filled, true strength 1) and without one (hollow, true strength 0).

We also trained every model on data with no context effect ($A = 0$). Then the probe should find nothing. Every model reports a small value, 0.03 to 0.10 (Figure 4.6), not exactly zero because the data are finite; the oracle itself reports 0.091 ± 0.011, because it fits all 256 entries of A freely. One-layer self-attention reports 0.033, the lowest, against 0.93 when the effect is present; mean-pool reports 0.097 against 0.95. So the models do not invent context effects, and for every model the two worlds are clearly separated.

4.2.4 How LCL-shaped are the models?

$\hat{A}$ is only a summary of what a model does, so we must ask how good the summary is. Self-attention (fidelity 0.91) and mean-pool (0.92) are about 90% LCL-shaped, while only 55–59% of the flat MLP's reaction to the slate follows the LCL form (Figure 4.7a). Metric C uses the true A directly and not the probe, and it gives the same picture (Figure 4.7b): self-attention under-reacts to a change of slate by about 8% (slope 0.92), mean-pool by about 6% (slope 0.94) and the flat MLP by 17–25%. This agrees with the size ratios of Figure 4.5c. The rule inside the free-form networks changes by 10–15% between usual and unusual slates (Figure 4.7c).

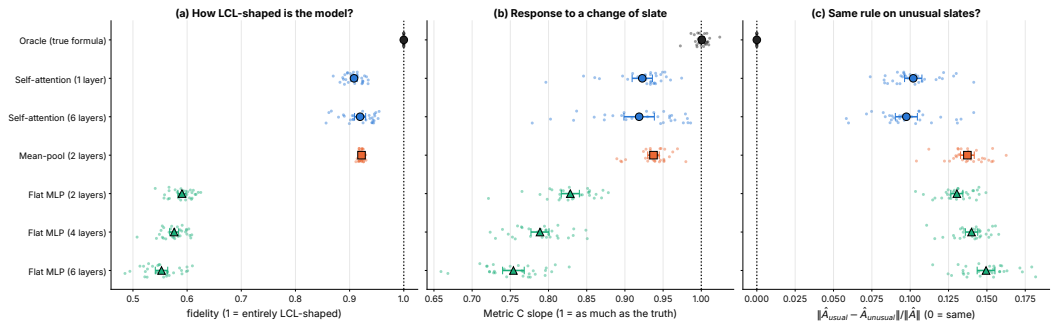


Figure 4.7: (a) Fidelity: share of the model's context behaviour explained by the LCL form. (b) Metric C slope: the model's change in preference when the slate changes, compared with the true change. (c) Invariance: change of $\hat{A}$ between usual and unusual slates (0 means the same rule).

So they are close to the LCL but not exactly LCL; self-attention is the most stable of them (0.10 against 0.14 for mean-pool).

4.2.5 Does accuracy imply recovery?

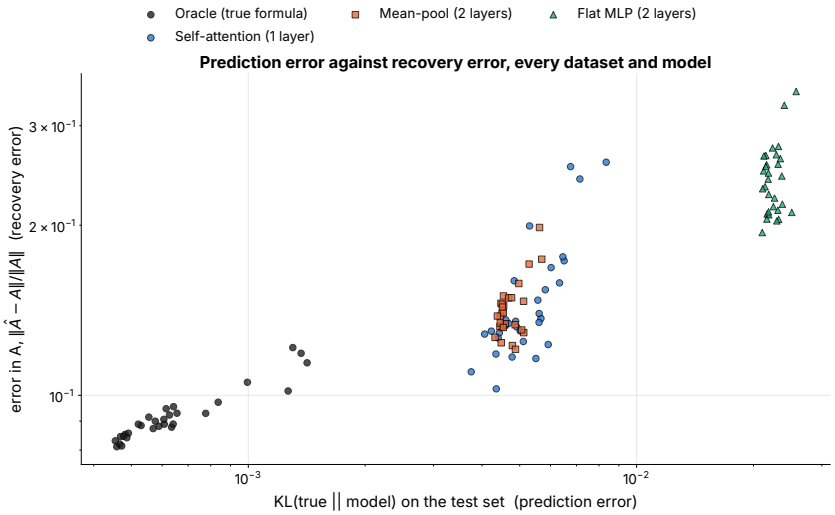


Figure 4.8: Prediction error (KL to the true probabilities) against recovery error of A, for every dataset and model (log scales).

Across architectures, prediction and recovery go together (Figure 4.8): the models that predict better also recover A better, with a correlation of 0.93 on the log scale. Inside one architecture the link is weaker: 0.79 for one-layer self-attention, 0.62 for mean-pool and 0.33 for the flat MLP. So the prediction error orders the architectures correctly, but it does not fix the recovery error of one single trained model.

Takeaway (RQ1). Self-attention that predicts well has also learned the context rule: its $\hat{A}$ has the right pattern (cosine 0.99), about 93% of the true strength and no false positives. Its error is still 1.5 to 1.6 times the data floor. We do *not* claim that it is better than mean-pool: the two are within noise in the error of A and mean-pool is slightly better in KL. The clear difference is to the flat MLP, which sees the same sessions but predicts and recovers much worse.

4.3 WHAT DECIDES RECOVERY (RQ2)

Each subsection changes one factor of Table 3.3 and keeps the rest at the reference setting.

4.3.1 Training-Set Size (E2)

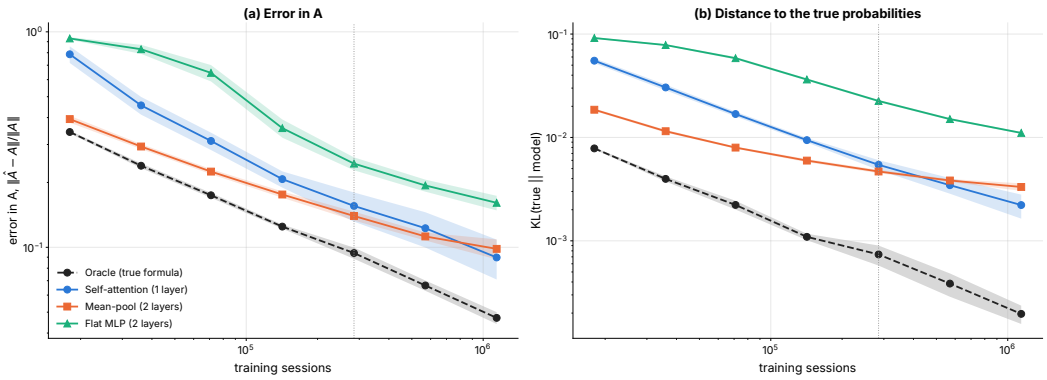


Figure 4.9: Recovery error of A (a) and KL to the true probabilities (b) against the number of training sessions, both on log scales (15 datasets per point). Dotted line: reference size.

The oracle's error falls on a straight line in the log–log plot, as expected for a model with the correct form (Figure 4.9). Mean-pool is close to the oracle when data are few (0.39 against 0.34 at 18,000 sessions) but improves more slowly when data are many (0.098 against 0.047 at 1.14 million). This points to a small bias: with ReLU layers it can only approximate the product of an item with the slate average. Self-attention starts far behind (0.79 at 18,000 sessions) but keeps improving. In KL it reaches mean-pool at about 570,000 sessions and is clearly below it at 1.14 million. In the error of A the two are tied at 1.14 million (0.090 against 0.098, overlapping

bands). The flat MLP needs about four times more data than self-attention for the same error (0.161 at 1.14 million against 0.156 for self-attention at 285,000; 0.194 at 570,000 against 0.207 for self-attention at 142,000).

Takeaway. Self-attention is the most data-hungry of the structured models, but it keeps improving with data and catches up with mean-pool at the largest sizes.

4.3.2 Rank of the Context-Effect Matrix (E3)

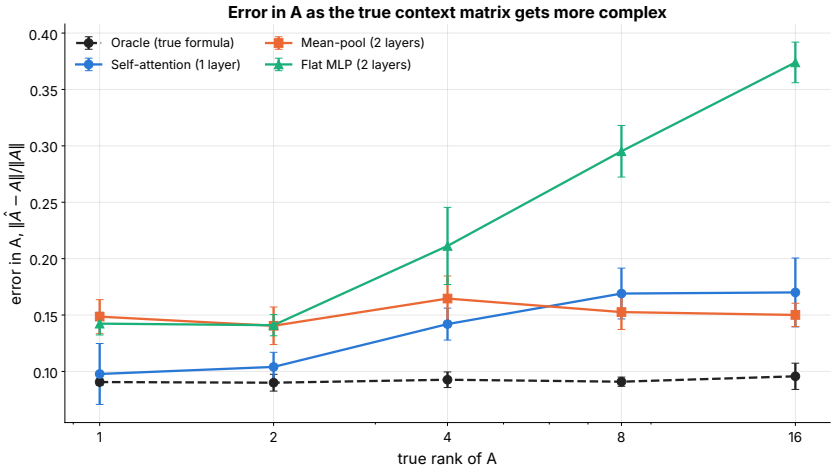


Figure 4.10: Recovery error of A against the true rank of A for the full-size models.

The oracle stays at about 0.09 for every rank (Figure 4.10). Self-attention is best when A is simple: at rank 1 and 2 it is almost at the oracle (0.098 and 0.104) and clearly better than mean-pool (0.149 and 0.140). Its error grows to 0.142 at rank 4 and 0.17 at ranks 8 and 16, where mean-pool (0.15 to 0.153) is slightly better. Mean-pool does not depend on the rank (0.14 to 0.17). The flat MLP gets worse fastest, from 0.14 at rank 1 to 0.37 at rank 16. The small attention models with head size below the rank are in Section 3.3.3: in the whole grid of true rank and head size, they stay within 0.006 cosine of the best possible matrix for their size, so their only limit is representation.

Takeaway. Self-attention recovers low-rank context effects almost as well as the oracle, and its error grows slowly as the rank grows.

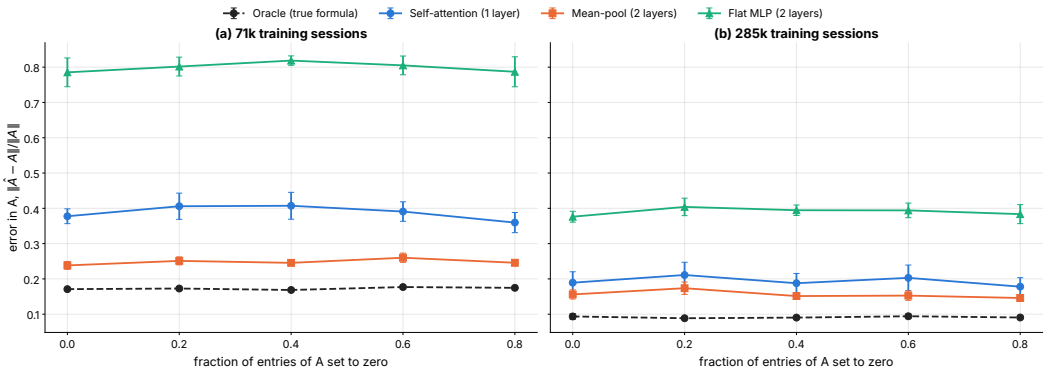


Figure 4.11: Recovery error of A against the fraction of entries of A set to zero (rank 16), at 71,000 (a) and 285,000 (b) training sessions.

4.3.3 Sparsity of the Context-Effect Matrix (E4)

Sparsity hardly changes recovery for any model at either data size, and the order of the models stays the same (Figure 4.11). This is what we expect, because no model has anything (for example an L_1 penalty) that uses zeros, and zeros do not make A harder to learn. No model finds exact zeros: on the entries that are truly zero (285,000 sessions), the error of self-attention and mean-pool is about 1.5 to 2.1 times the oracle's, and that of the flat MLP about 2.2 to 2.9 times.

Takeaway. Sparsity of A neither helps nor hurts recovery.

4.3.4 Number of Features (E4)

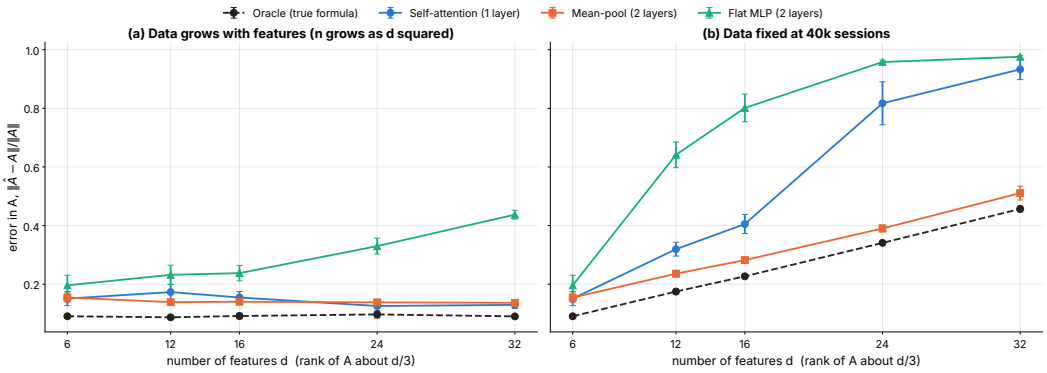


Figure 4.12: Recovery error of A against the number of features d (rank about $d/3$). (a) Training data grow as d^2 (40,000 sessions at $d = 6$ up to 1.14 million at $d = 32$). (b) Training data fixed at 40,000 sessions. 10 datasets per point.

When the data grow with the number of entries of A, self-attention and mean-pool

keep an almost constant error up to 32 features (0.13–0.17 and 0.14–0.15), and the oracle stays at 0.09 (Figure 4.12a). The flat MLP gets worse steadily, from 0.20 to 0.44. When the data are fixed at 40,000 sessions, all models get worse as d grows, because there are more entries to estimate (Figure 4.12b). Mean-pool stays close to the oracle (0.51 against 0.46 at 32 features), but self-attention rises to 0.93, close to the flat MLP (0.98).

Takeaway. Self-attention scales to larger feature spaces when the data scale with them. With little data per entry of A, it is the most data-hungry structured model.

4.3.5 Depth of the Network (E5)

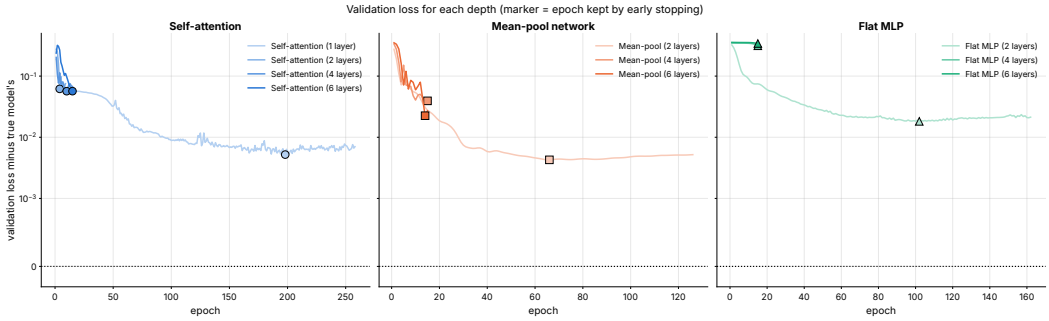


Figure 4.13: Validation loss above the true model's, for each depth of the self-attention, mean-pool and flat MLP models (lighter colour: fewer layers). Markers: epoch kept by early stopping.

Deeper models train differently (Figure 4.13). Deeper self-attention needs much longer: on this dataset the 6-layer model was kept at epoch 472 and trained until the 500-epoch limit, and its curve is noisy. Deeper flat MLPs also need hundreds of epochs, while the mean-pool curves look alike for all depths and start to rise after about 60 epochs. This is why some deep models hit the epoch limit (Section 3.4): 3 of 30 six-layer self-attention runs and 13 of the 60 four- and six-layer flat MLP runs in the reference experiment.

For recovery, self-attention and mean-pool show no clear trend with depth at either data size, since the differences are within the confidence intervals (Figure 4.14). Depth does not rescue the flat MLP: at 71,000 sessions more layers help it a little (0.65 to 0.55), at 285,000 sessions they hurt (0.245 to 0.32). The fair depth

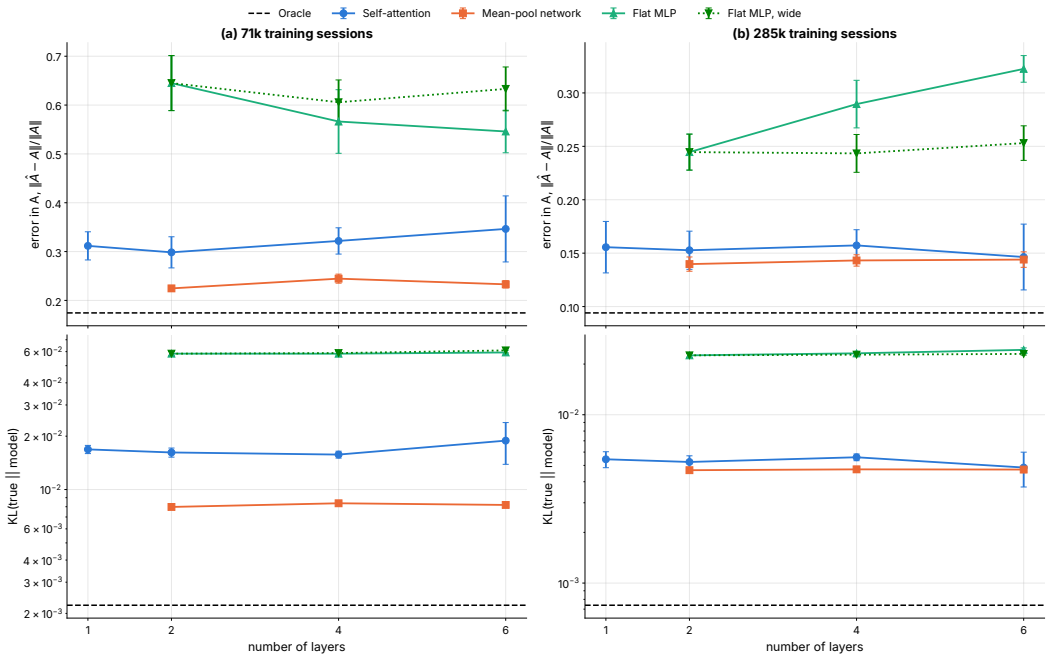


Figure 4.14: Recovery error of A (top) and KL to the true probabilities (bottom, log scale) against the number of layers, at 71,000 (a) and 285,000 (b) sessions (15 datasets per point). The flat MLP is re-matched to the size of one-layer self-attention at every depth; the wide flat MLP keeps its width. Dashed line: oracle.

comparison is the wide flat MLP against mean-pool, because both keep their width and grow with depth (the narrow flat MLP gets thinner as it gets deeper). The wide flat MLP does not get better with depth (0.24 to 0.25 at 285,000 sessions), and it is worse than mean-pool at every depth: about 1.7 times in the error of A and 4.8 to 4.9 times in KL at 285,000 sessions, and 2.5 to 2.9 times and 7 to 7.4 times at 71,000 sessions. At two layers, where mean-pool and the flat MLP have the same size, the gap is already 1.75 times, so it is not caused by size. (The mean-pool network is slightly larger than the wide flat MLP at 4 and 6 layers, 22,000 and 35,000 against 17,000 and 25,000 parameters.) At every depth and data size the narrow and the wide flat MLP are about 1.6 to 2.9 times worse than self-attention and mean-pool in the error of A. The depth sweep is not size-matched for all families (Table 3.1); but since extra depth, and with it extra parameters, did not help self-attention or mean-pool, this cannot explain the result for them.

Takeaway. Depth is not what makes a network learn the context effect. The structure is: treating the slate as a set and comparing each item with the rest of it.

4.3.6 Random Initialisation (E6)

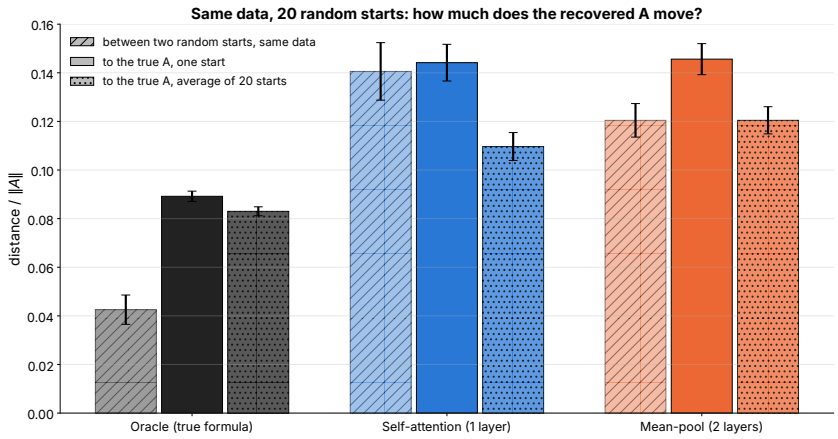


Figure 4.15: Ten datasets, each trained from 20 random initialisations. Hatched: mean distance between the $\hat{A}$ of two initialisations on the same data. Solid: mean error of one initialisation. Dotted: error of the average of the 20 $\hat{A}$'s. All divided by $\|A\|$.

For the oracle, two initialisations on the same data differ by only 0.04, much less than its error (0.09): its error comes almost completely from the finite data (Figure 4.15). For self-attention, two initialisations differ by 0.14, *as much as one of them differs from the truth* (0.14). So a large part of its error comes from the optimisation and not from the data. Averaging the 20 recovered matrices brings its error from 0.144 to 0.110, below the averaged mean-pool (0.120). Mean-pool shows the same effect more weakly (0.120 between initialisations, error 0.146). If we compare squared errors, averaging removes about 40% of the squared error of self-attention, 32% for mean-pool and 13% for the oracle.

Takeaway. About 40% of the remaining (squared) error of self-attention is due to the random initialisation, and averaging several trained models removes it.

4.3.7 Optimisation Settings (E7)

Over the whole tuning grid, self-attention stays at 0.15–0.17, mean-pool at 0.14–0.15 and the flat MLP at 0.24–0.26 (Figure 4.16a). The spread due to hyperparameters (about 0.02 at most) is much smaller than the gap between architectures, and the order never changes. Mean-pool improves a little with width (0.153, 0.142,

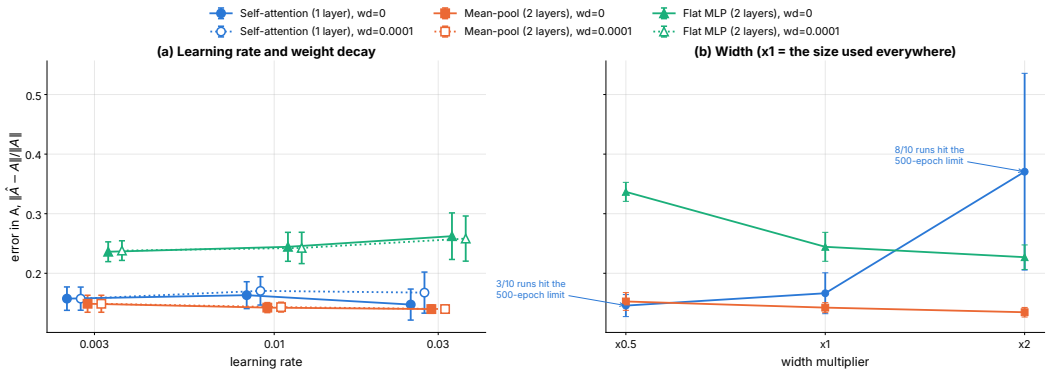


Figure 4.16: Recovery error of A (a) for every learning-rate and weight-decay setting (filled: no weight decay; hollow: 10^{-4}) and (b) for half, normal and double width (10 datasets per point).

0.135 for half, normal and double width), and the flat MLP improves more (0.34 to 0.23) but stays behind. Double-width self-attention gives a poor value (0.37). But 8 of its 10 runs were still improving when they reached the 500-epoch limit, so this point shows an unfinished training and not a lack of capacity; 3 of 10 half-width runs also hit the limit, but their error (0.146) is as good as the normal model's (0.167 on these datasets). decide: rerun double-width self-attention with a longer epoch budget, or keep it as reported (unconverged).

Takeaway. The conclusions do not depend on the learning rate, the weight decay or (up to the unconverged point) the width.

4.3.8 Recovery of the Preference Vector

The preference vector θ is the easy part. At the reference setting every model recovers it within 2–5% (Table 4.1), also the flat MLP, which recovers A worst. Its error falls with data for every model (Figure 4.17a): at 1.14 million sessions it is 1.3% for the oracle, 2.1% for mean-pool and 2.7% for the flat MLP, and 2.8% for self-attention. At the reference size, self-attention has the largest θ error of the four (4.1%), the reverse of its ranking for A. The rank and sparsity of A do not change $\hat{\theta}$ (b, c). With the data fixed at 40,000 sessions, the flat MLP's error jumps to 13–15% at 24 and 32 features, while self-attention and mean-pool stay below 9% (e). Depth changes $\hat{\theta}$ little (f).

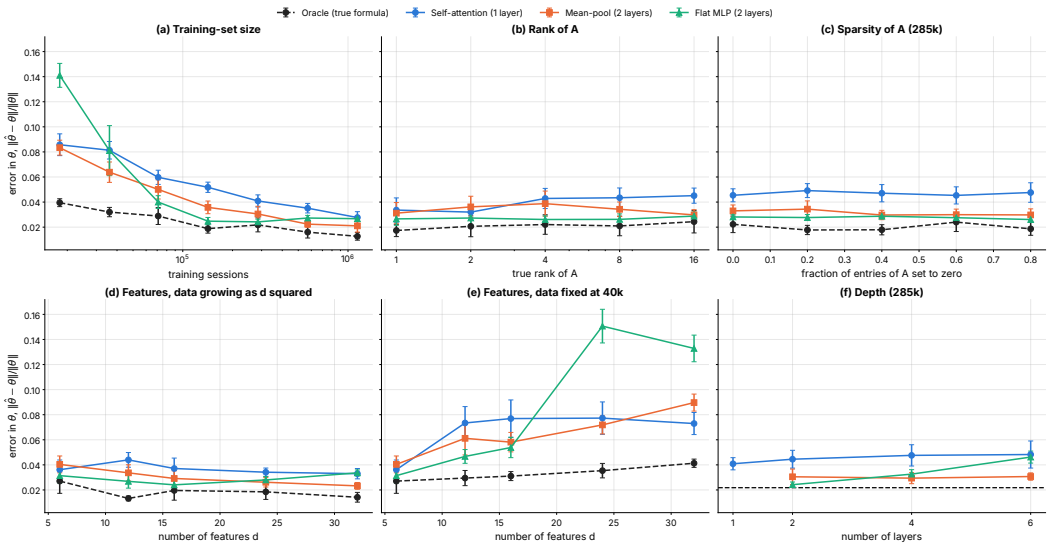


Figure 4.17: Relative error of $\hat{\theta}$ across the sweeps of this section: (a) training-set size, (b) rank of A, (c) sparsity, (d) features with data growing as d^2, (e) features with data fixed at 40,000, (f) depth.

Takeaway. Every model recovers θ well. The models differ in how well they learn the context effect, not the base preferences.

4.4 BEHAVIOUR ON SHIFTED CHOICE SETS (RQ3)

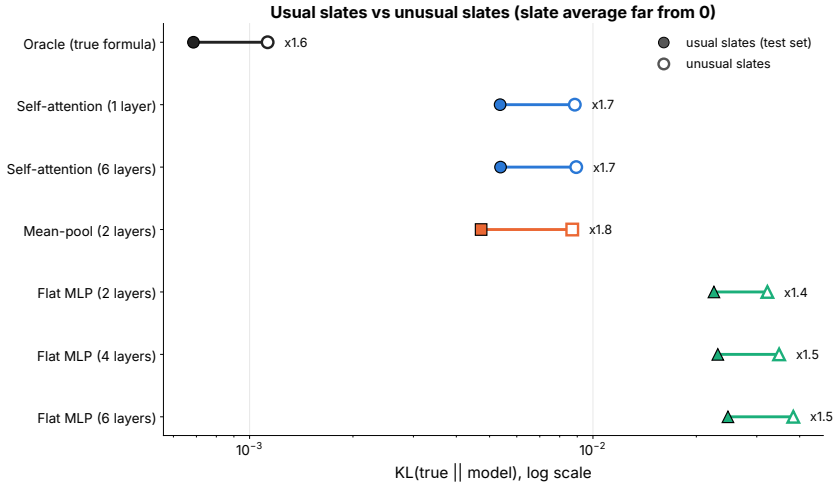


Figure 4.18: KL to the true probabilities on the usual test set (filled) and on the shifted set (hollow), 30 datasets. Numbers: ratio of the two.

The shifted set contains the 5% of slates whose average y_S is farthest from zero (Section 3.2.3). Every model is worse on them, including the oracle (1.6 times its

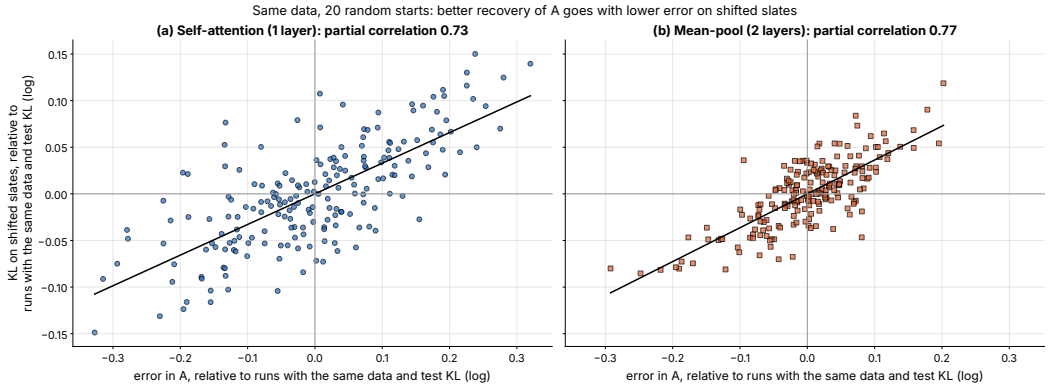


Figure 4.19: Same data, 20 random initialisations (10 datasets): error of $\hat{A}$ against KL on the shifted set, after removing what the dataset and the usual-test KL explain (added-variable plot, log scales).

usual KL; Figure 4.18). This is the general difficulty of estimating far from where most data lie, not a fault of one architecture. Self-attention and mean-pool get worse by about the same factor as the oracle (1.7 and 1.8), so they did not learn a rule that works only on typical slates. The flat MLP gets worse by a smaller factor (1.4 to 1.5), but from a level about four times higher.

Now we ask whether recovery matters *beyond* the fit on usual data. For this we use the initialisation experiment, where the data are the same and only the starting weights change. After removing what the dataset and the usual-test KL explain, runs that recovered A better still predict the shifted slates better (Figure 4.19): the partial correlation is 0.73 for self-attention and 0.77 for mean-pool. In logs, a 10% larger error in A goes with about 3 to 3.5% larger KL on shifted slates at the same usual KL. The effect is moderate, because the usual KL already explains most of the shifted KL (raw correlation 0.89 and 0.91), and the shift is mild because the shifted slates still come from the same Gaussian.

Takeaway (RQ3). Among models that fit the usual data equally well, the ones that recovered the context matrix better also generalise better to unusual choice sets.

4.5 ATTENTION WEIGHTS VERSUS BEHAVIOURAL PROBING

For a bilinear attention layer the context matrix can be read from the weights, $A = W_q^\top W_k$. The probe check V2 (Section 3.6) used exactly such a layer, and the

probe returned its matrix to about 10^{-15}, so for this model class the two readings are the same.

For the transformer there is no single matrix in the weights that plays the role of A. The effect of the slate on the score of an item goes through the input embedding, the softmax of the attention weights, the value and output projections of two heads, layer normalisation, the residual path and the feed-forward block. The attention weights α_{ij} say how much item i uses item j, not how the slate changes the utility of the item, and many different weight settings give the same behaviour (Section 4.3.6 shows that different initialisations give quite different $\hat{A}$ on the same data). For this reason all comparisons in this chapter use the behavioural probe, which measures what the model does and not how it is written. ask the advisor whether a direct comparison of attention maps with $\hat{A}$ is wanted; it needs a new short run that saves trained weights.

4.6 SUMMARY OF FINDINGS

RQ1. Prediction and recovery go together across architectures. Self-attention keeps about 92% of the value of the context and recovers A with the right pattern, 93% of its strength and no false positives; its error is 1.5 to 1.6 times the data floor. It is not better than mean-pool (within noise in the error of A, slightly worse in KL), and clearly better than the flat MLP.

RQ2. Recovery depends on the amount of data, the rank of A and the number of features. Sparsity, depth, learning rate and weight decay do not change it, and neither does width, except for an unconverged double-width model. About 40% of the squared error of self-attention comes from the random initialisation.

RQ3. Among models that fit the usual data equally well, better recovery of A goes with better predictions on unusual choice sets.

Limitations.

- The data are synthetic: Gaussian item features, slate size five and a context

effect that is exactly linear in the slate mean. Real choice data may differ in all of these.

- The shifted set is a mild shift taken from the same Gaussian; a stronger shift may separate the models more.
- The training curves and accuracies come from one dataset; all other results average 10 to 30 datasets.
- 64 of 5,020 trainings hit the epoch limit, mostly deep flat MLPs and self-attention models; double-width self-attention did not converge.
- Several models chose the largest learning rate of the tuning grid, but Section 4.3.7 suggests that this does not change the conclusions.
- Dropout was not used or tested (Section 3.7).

Future work. Applying the probe to real choice data; slates of different sizes and non-LCL context effects; ensembles of self-attention models, since the initialisation experiment suggests they recover A better; longer training for wider and deeper models; and a direct analysis of transformer weights next to the probe.

CHAPTER A

PLANTED TRUTH AND HYPERPARAMETER TUNING

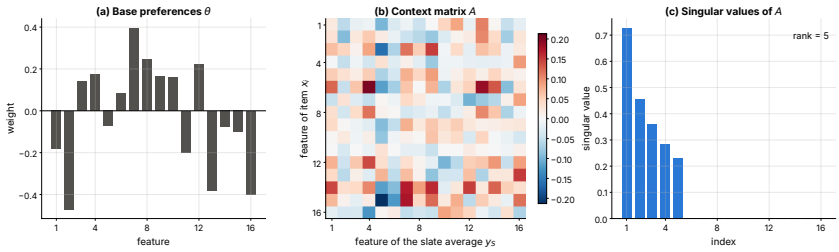


Figure A.1: Planted truth of the first reference dataset (seed 101). (a) Preference vector θ. (b) Context matrix A (red positive, blue negative); entry (k, l) is the effect of feature l of the slate mean on feature k of an item. (c) Singular values of A: exactly five are non-zero.

Table A.1 gives the learning rate and weight decay chosen for each model (lowest mean validation loss over two datasets, seeds 1 and 2). Figure A.2 shows the whole grid. The flat MLPs are the most sensitive to the choice (the worst pair is up to 0.0077 nats above the best for 6 layers and 0.0123 for wide 6 layers) and prefer the smallest learning rate.

Table A.1: Chosen learning rate and weight decay.

Model	Learning rate	Weight decay
Oracle LCL	0.03	10^{-4}
Bilinear attention	0.03	0
Softmax head	0.03	10^{-4}
Self-attention, 1 / 2 / 4 / 6 layers	0.03 / 0.03 / 0.01 / 0.03	10^{-4} / 10^{-4} / 0 / 10^{-4}
Mean-pool, 2 / 4 / 6 layers	0.01	0
Flat MLP, 2 layers	0.01	0
Flat MLP, 4 / 6 layers	0.003	0
Wide flat MLP, 4 / 6 layers	0.003	0

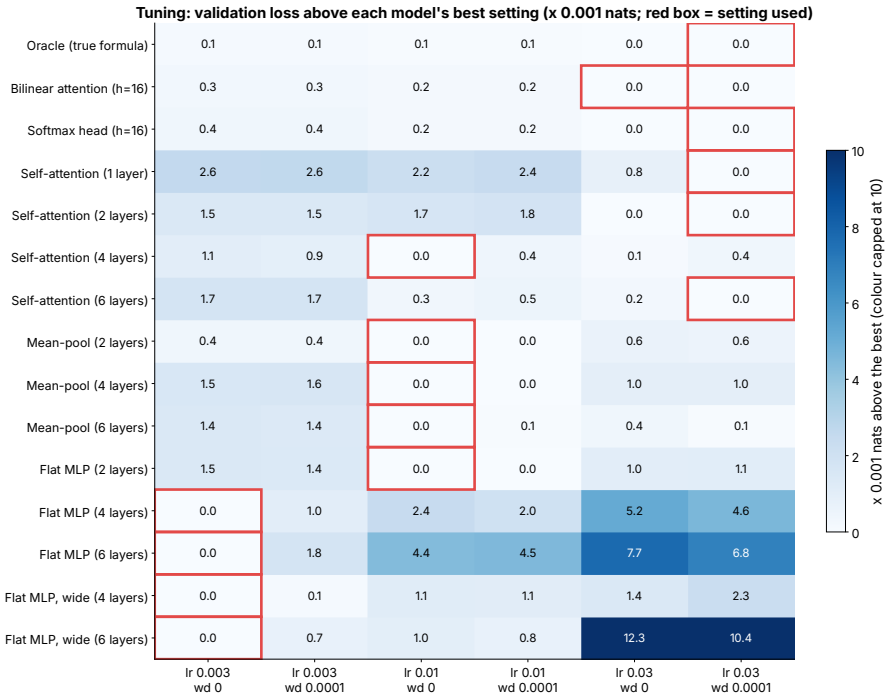


Figure A.2: Hyperparameter tuning. Each cell is the validation loss of one learning-rate and weight-decay setting minus the best setting of that model, in thousandths of a nat (mean of two datasets). The red box marks the setting used.

BIBLIOGRAPHY

Christopher J. C. Burges. From ranknet to lambdarank to lambdamart: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

Gerard Debreu. Review of R. D. Luce, Individual Choice Behavior: A Theoretical Analysis. *American Economic Review*, 50(1):186–188, 1960.

Robert L. Goldstone, Douglas L. Medin, and Jamin Halberstadt. Similarity in context. *Memory & Cognition*, 25(2):237–255, 1997.

Joel Huber, John W. Payne, and Christopher Puto. Adding asymmetrically dominated alternatives: Violations of regularity and the similarity hypothesis. *Journal of Consumer Research*, 9(1):90–98, 1982.

R. Duncan Luce. *Individual Choice Behavior: A Theoretical Analysis*. Wiley, 1959.

Daniel McFadden. Conditional logit analysis of qualitative choice behavior. In Paul Zarembka, editor, *Frontiers in Econometrics*. Academic Press, 1974.

Karlson Pfannschmidt, Pritha Gupta, and Eyke Hüllermeier. Learning context-dependent choice functions. *International Journal of Approximate Reasoning*, 2022.

Przemysław Pobrotyn, Tomasz Bartczak, Mikołaj Synowiec, Radosław Białobrzeski, and Jarosław Bojar. Context-aware learning to rank with self-attention. *arXiv preprint arXiv:2005.10084*, 2020.

Arjun Seshadri, Alexander Peysakhovich, and Johan Ugander. Discovering context effects from raw choice data. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

Itamar Simonson. Choice in context: Tradeoff contrast and extremeness aversion. *Journal of Consumer Research*, 16(2):158–174, 1989.

Kiran Tomlinson and Austin R. Benson. Learning interpretable feature context effects in discrete choice. In *Proceedings of the ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2021.

Amos Tversky. Elimination by aspects: A theory of choice. *Psychological Review*, 79(4):281–299, 1972.

Amos Tversky. Features of similarity. *Psychological Review*, 84(4):327–352, 1977.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones,

Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.